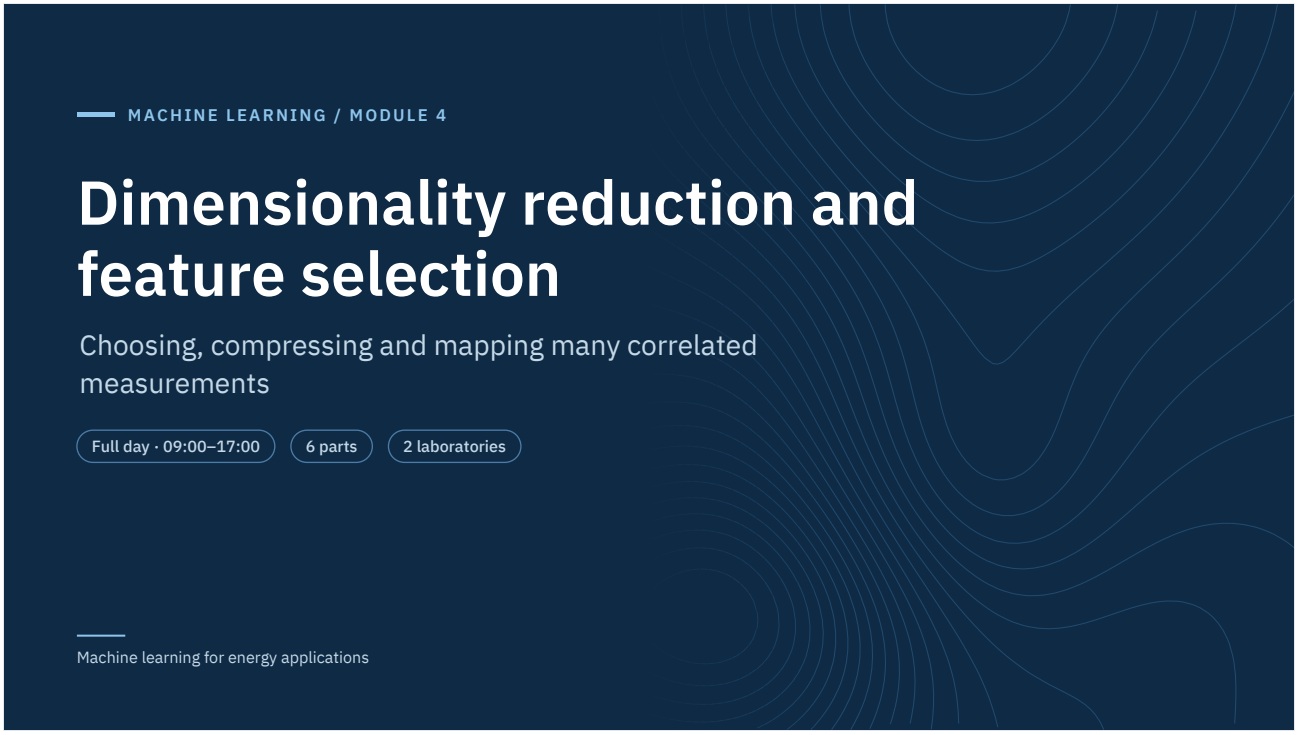
Dimensionality reduction and feature selection



Welcome the group and connect to earlier modules: Module 1 built pipelines and honest splits, Module 2 added ensembles, interpretation and the leakage demonstration, Module 3 clustered unlabelled data. Today asks what to do when there are too many columns: which ones to keep (feature selection), how to summarize them (PCA and its relatives), and how to draw maps of neighbourhoods (t-SNE, UMAP) without over-reading them.

Prerequisites: standardization, covariance, cross-validation and pipelines. Everything runs in scikit-learn. UMAP needs the optional umap-learn package, listed in requirements-labs-optional.txt, so its slides are conceptual and its notebook is optional.

Keep one distinction visible all day: an exploratory picture is not a production feature transform, and neither is evidence for a physical claim until it has been checked in original units. [2 min]

Today's plan



Walk through the day. Part 1 (why reduce, 09:00–09:40, including this introduction) and Part 2 (feature selection, 09:40–10:30) form one morning block. Part 3 (PCA in depth, 10:45–11:40) leads into Lab 1, which applies it to drilling data until lunch; its debrief opens the afternoon.

After the debrief (13:40–13:55), Part 4 covers kernel PCA, LDA, ICA, NMF and an autoencoder preview (13:55–14:25). Part 5 covers manifold learning, t-SNE, UMAP and trustworthiness (14:25–15:15). Lab 2 (15:30–16:30) offers two routes: t-SNE on production data, or a method comparison on petrophysical data.

The last block combines the Lab 2 debrief, Part 6 (reduction in practice, with three energy applications) and the exit check. If time runs short, shorten Part 6 to the decision table and one application; keep the exit check. [2 min]

What you will be able to do

LEARNING OUTCOMES

What you will be able to do

By the end of the day you will be able to:

1. Explain why many features weaken distance-based methods, and when to reduce.
2. Select features with filter, wrapper and embedded methods inside cross-validation.
3. Compute and validate a PCA: scaling, loadings, number of components, reconstruction error.
4. Choose kernel PCA, LDA, ICA or NMF for a stated goal.
5. Build t-SNE, Isomap and UMAP maps and state what they cannot show.
6. Judge a representation by trustworthiness, stability and held-out task score.

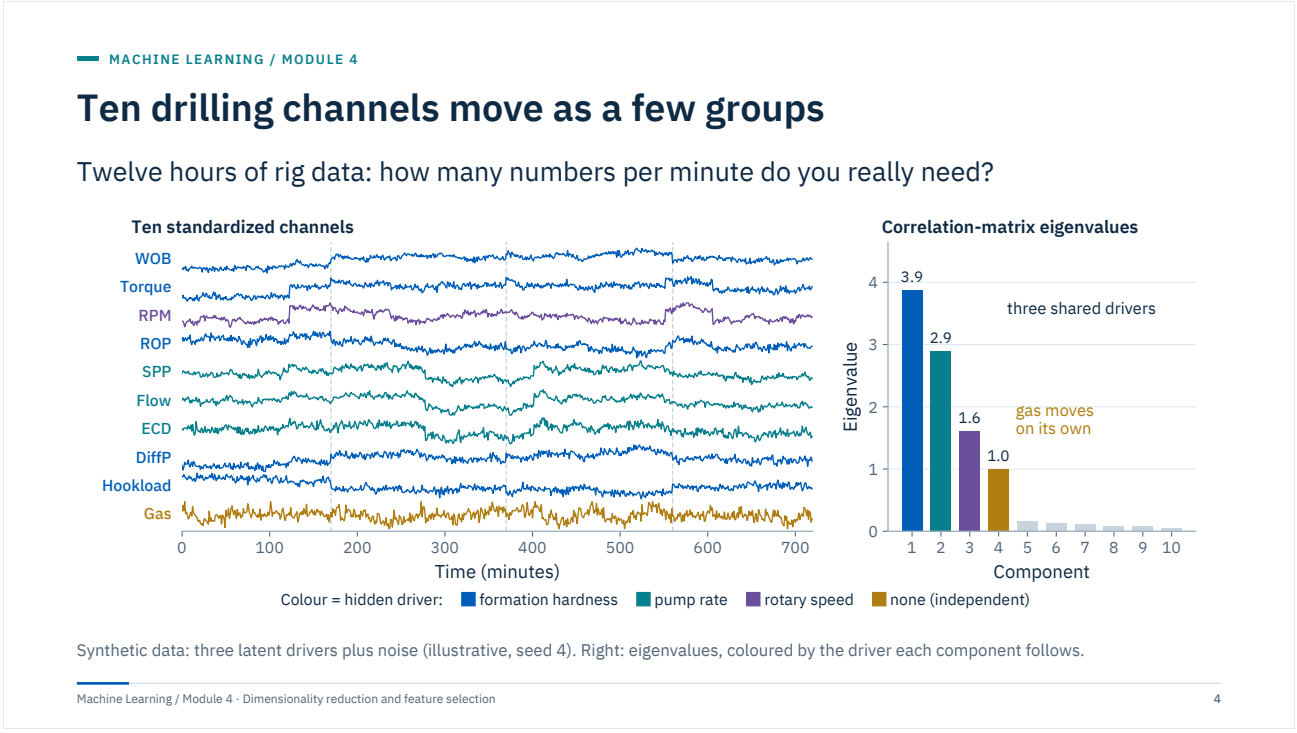
Machine Learning / Module 4 · Dimensionality reduction and feature selection 3

Read the outcomes aloud and point out that each one is observable: by the end of the labs every learner should have produced a scree plot with a justified number of components, a loadings table read in words, and at least one nonlinear map with a trustworthiness score and a stated limitation.

Outcome 2 is the one most often done wrongly in practice: selection that looks at the target before the split. Outcome 6 is the thread through the afternoon: a representation is judged by evidence (neighbourhood preservation, stability across seeds and folds, and task performance on held-out wells), not by how clean the picture looks.

Ask the room which outcome matters most for their own work, and note answers on the whiteboard to revisit at the exit check. [2 min]

Ten drilling channels move as a few groups



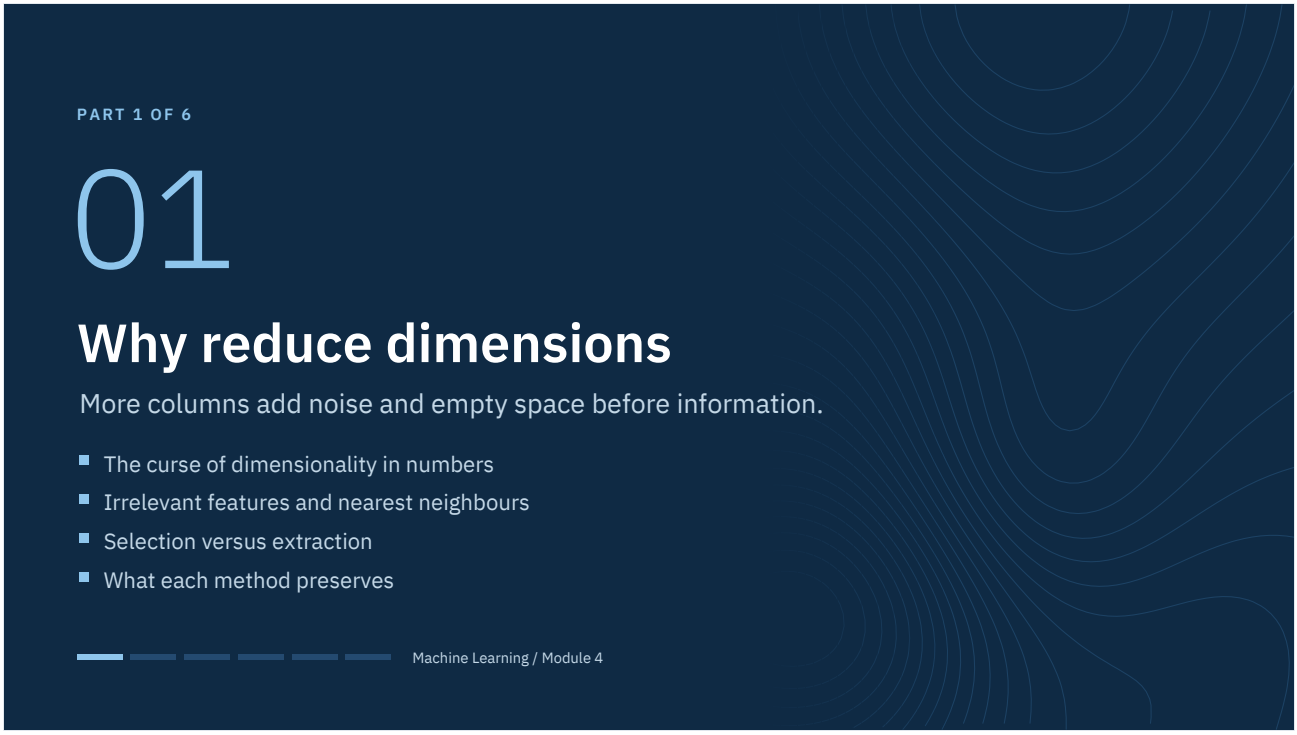
Motivating case [5 min]. Before discussing the right panel, ask: "Ten channels arrive every minute. How many numbers per minute would you store to keep most of the information?" Let pairs guess.

Reading the left panel: WOB, torque, differential pressure, hook load and ROP step together at the formation changes (dashed lines); SPP, flow and ECD follow the pump rate; RPM follows its own setpoints; gas wanders independently. Colours mark the hidden driver that generated each channel; with real data you would not know this in advance.

The right panel quantifies it: three large eigenvalues (3.9, 2.9 and 1.6) for three shared drivers, one eigenvalue of 1.0 for the independent gas channel, and six small ones that are mostly noise. The bar colours anticipate the loadings slide in Part 3: component 1 follows formation hardness, 2 the pump rate, 3 the rotary speed and 4 the gas channel. Four numbers per minute keep about 94% of the standardized variance.

Plant two questions for the day. Is gas "noise" because it is independent? No, it may be the safety-critical channel. Does 94% of variance mean 94% of what matters for a target? Not necessarily; Part 3 tests this directly.

Why reduce dimensions



Part 1 runs about 30 minutes. The aim is motivation with numbers rather than slogans: why high-dimensional spaces behave strangely, why that hurts the distance-based methods from Module 1 (KNN, SVM with RBF kernels) and Module 3 (k-means, DBSCAN), and the two families of remedies: keep a subset of the original columns, or build new ones. The part ends with a short matching exercise on energy tasks. [1 min]

Reduction serves four different goals

WHY REDUCE DIMENSIONS

Reduction serves four different goals

Goal	Useful output	Evidence to inspect
Visualization	Two- or three-dimensional coordinates	Neighbourhood preservation
Compression	Lower-dimensional representation	Reconstruction error
Denoising	Restricted representation	Signal preservation
Prediction	Features for a downstream estimator	Held-out task performance

Feature selection keeps original variables; feature extraction builds new ones.

◆ Name the goal before choosing a method: the goal decides which evidence counts.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 6

Start from the goal, because the goal decides the evidence. A visualization is judged by whether neighbours in the map were neighbours in the data. A compression is judged by reconstruction error, ideally per channel and in engineering units. Denoising is judged by whether the signal you care about survives. A predictive reduction is judged only by held-out task performance compared with no reduction on the same folds.

Feature selection retains original variables, while feature extraction builds new variables. Reduction can discard useful low-variance information, and a beautiful embedding is not automatically a better predictive representation.

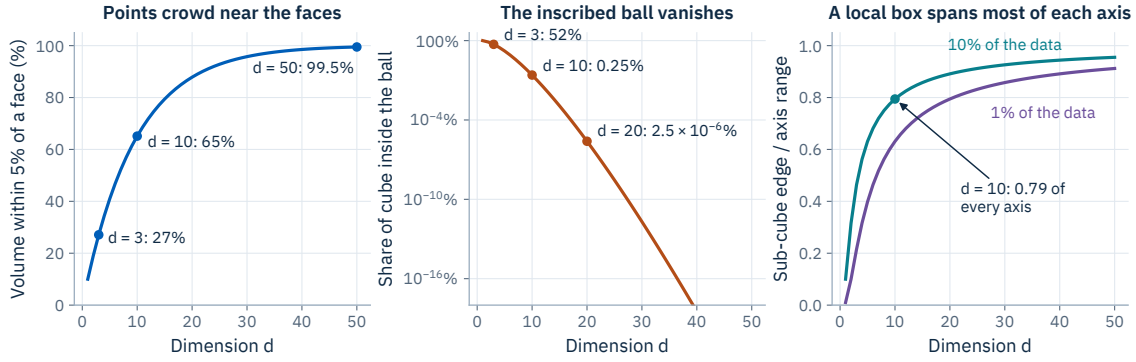
Ask: which of these goals have you pursued with PCA without stating it? Most people have used PCA "for prediction" while only checking explained variance, which is compression evidence. [3 min]

High-dimensional volume concentrates near the edges

WHY REDUCE DIMENSIONS

High-dimensional volume concentrates near the edges

Three exact calculations for points spread uniformly in a unit cube.



Exact formulas: shell volume 1 − 0.9^d; inscribed-ball share π^(d/2) / Γ(d/2 + 1) × 0.5^d; sub-cube edge r^(1/d) for a fraction r of the data.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 7

Walk through the three panels [4 min].

Left: the share of the cube within 5% of a face is 1 − 0.9^d, because the inner cube has side 0.9. It is 27% in 3-D, 65% in 10-D and 99.5% in 50-D, so uniform points sit near the boundary.

Middle: the inscribed ball holds 52% of the cube in 3-D, 0.25% in 10-D and about 2.5 × 10^{-6} % in 20-D. Almost all the volume sits in the corners.

Right: a sub-cube holding 10% of uniform data needs edge 0.1^(1/d): 0.79 of every axis in 10-D. A "local" neighbourhood is no longer local (Hastie, Tibshirani and Friedman, section 2.5).

Caveat: these results assume uniform, independent coordinates. Real measurements are correlated and concentrate near lower-dimensional structure, which is exactly why reduction can work. The curse bites hardest when many columns are irrelevant noise. Transition: how much data would we need to fill the space?

Keeping neighbour density needs exponentially more data



The arithmetic is exact: 10 grid points per axis in d dimensions is 10^d points, so the same neighbour spacing needs 10 samples in 1-D, 100,000 in 5-D and 10^10 (ten billion) in 10-D [2 min].

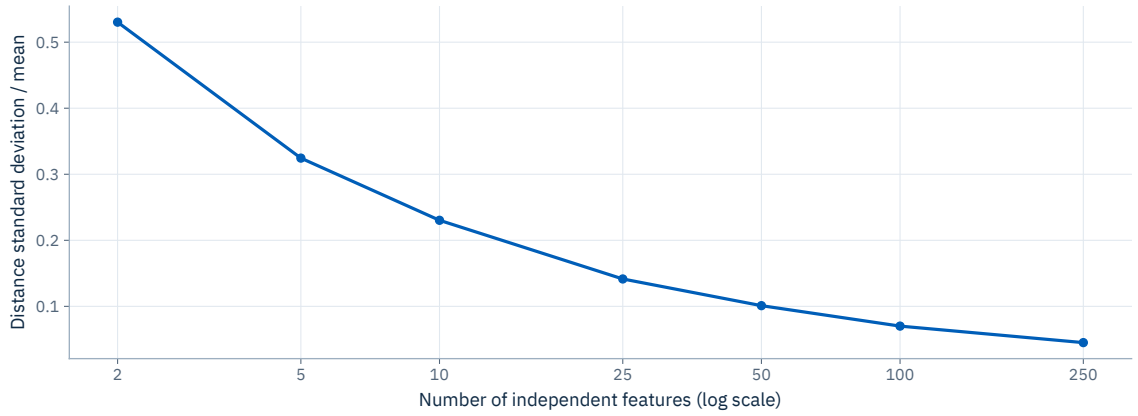
Ask the room how many rows their largest drilling or log table has. Even millions of rows are tiny against 10^10, yet models work. Why? Because the ten channels are not ten independent directions: in the motivating case, four components carried about 94% of the standardized variance. The effective dimension is what matters, and reduction methods try to find it.

Two practical consequences. Distance-based models (KNN, RBF kernels, k-means, DBSCAN) are the most exposed. And adding weakly relevant columns is not free: each one spreads the data further. The next two slides show both effects numerically.

Distances can become less distinctive

WHY REDUCE DIMENSIONS

Distances can become less distinctive



Simulation with independent standard-normal features. Relative pairwise-distance spread shrinks as dimension grows.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 9

The curve shows the standard deviation of pairwise distances divided by their mean. With independent standard-normal features it falls from 0.53 in 2-D to 0.23 in 10-D and 0.045 in 250-D: nearest and farthest neighbours become almost equally far away [3 min].

The phenomenon depends on the data distribution and the metric. Real data may lie on a lower-dimensional manifold, where concentration is much weaker.

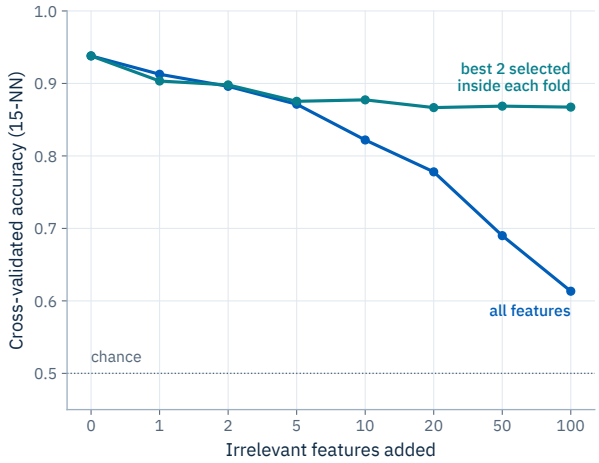
Explain that simply adding noisy features can weaken nearest-neighbour methods even when the original signal remains: the distance is a sum over features, and every irrelevant feature adds variance to it without adding information. Ask: which of your datasets contain dozens of "just in case" columns? Transition: the next slide measures the damage directly.

Irrelevant features drown a nearest-neighbour signal

WHY REDUCE DIMENSIONS

Irrelevant features drown a nearest-neighbour signal

- Two informative features; pure-noise columns added in blocks
- 15-NN accuracy falls from 0.94 to 0.61 with 100 noise columns
- Selecting the best 2 features inside each fold holds 0.87



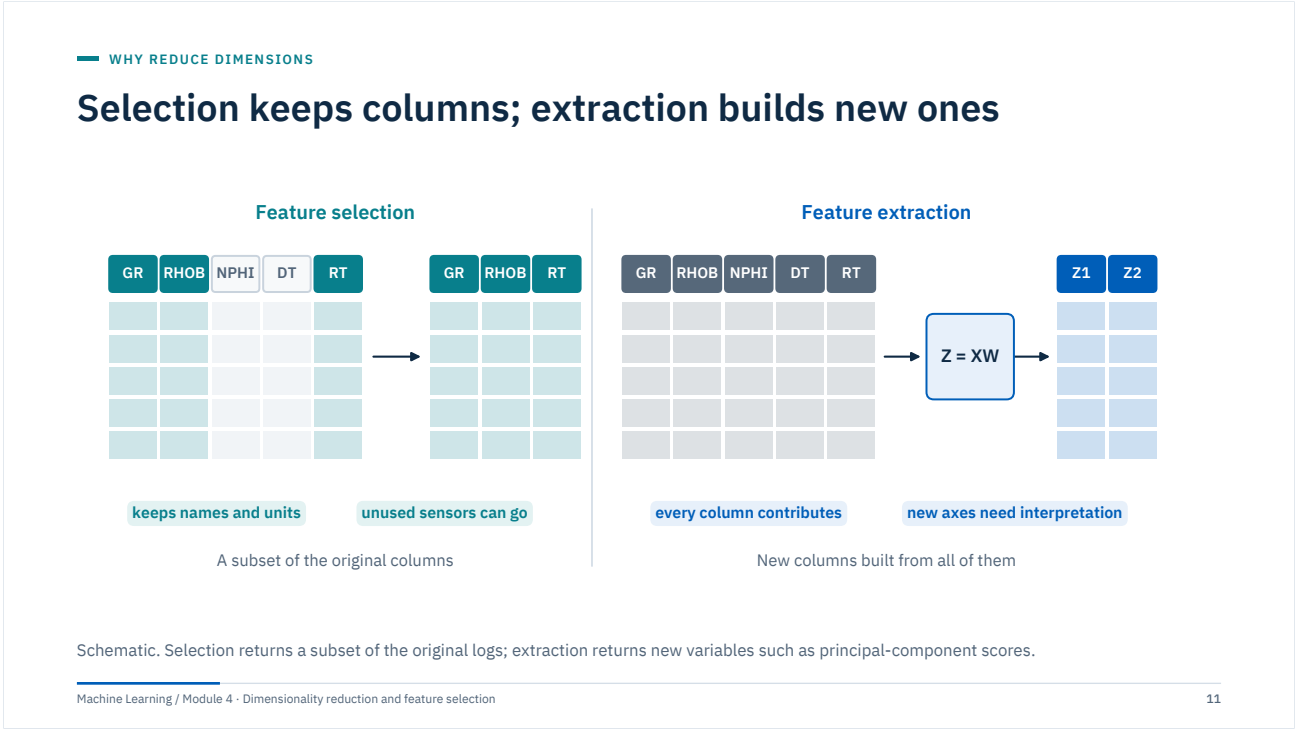
Synthetic two-class data (seeds 0–4); 5-fold CV averaged over five datasets.

The experiment: two informative features separate two classes; then 1 to 100 pure-noise columns are appended. The blue line is a standardized 15-nearest-neighbour classifier on all columns; its cross-validated accuracy falls from 0.94 to 0.61, close to the 0.5 chance line. The teal line adds SelectKBest (F-test, k = 2) inside each training fold and stays near 0.87 [3 min].

Why the selector does not fully recover 0.94: one of the two informative features carries little class signal on its own, and among 100 noise columns the F-test always finds one that scores higher (in all 25 folds). A univariate filter cannot see a feature that only helps jointly, a blind spot Part 2 returns to.

Not every model is this fragile: regularized linear models and tree ensembles resist irrelevant columns better, but none are immune, and all pay in variance and compute. Ask: would you expect a random forest curve to fall as fast? (No: on the same data a forest drops only from 0.94 to 0.88 with 100 noise columns, but it still drifts down.) Transition: two families of remedies.

Selection keeps columns; extraction builds new ones



Selection keeps a subset of the original columns: GR, RHOB and RT here. Names and units survive, so a petrophysicist can read the model, and unused sensors can be removed from the next acquisition programme. The cost: if a dropped column carried a little unique information, it is gone.

Extraction builds new columns from all of them, Z = XW for linear methods such as PCA. Every original column contributes a little, which is good when many correlated channels each carry the same signal with independent noise. The cost: new axes must be interpreted, and all original sensors are still needed to compute them.

Hybrids are common: remove dead or redundant channels first, then summarize correlated blocks with PCA. Ask: for a new-rig sensor specification, which would you use? (Selection, because extraction still needs every sensor.) [3 min]

Reduction methods preserve different things

WHY REDUCE DIMENSIONS

Reduction methods preserve different things

Method	Preserves	Uses labels	New observations
PCA, kernel PCA	Variance, linear or in a kernel space	No	`transform()`
LDA	Separation of known classes	Yes	`transform()`
Isomap, LLE	Geodesic or local linear structure	No	Approximate `transform()`
t-SNE	Local neighbourhoods	No	None in sklearn TSNE
UMAP	Neighbourhood graph	Optional	Fitted `transform()`

Parameter settings and preprocessing remain part of the method.

This table is the map of the day [3 min]. PCA and kernel PCA preserve variance; LDA preserves separation between labelled classes; Isomap and LLE preserve geodesic distances or local linear reconstructions; t-SNE preserves local neighbourhoods; UMAP preserves a weighted neighbourhood graph. ICA and NMF (Part 4) change the objective to independence and non-negative parts.

The last column matters for production use. sklearn's TSNE has no transform method for new rows: t-SNE extensions may support out-of-sample insertion, which is different from sklearn's standard API. Isomap and LLE provide approximate out-of-sample transforms. UMAP provides a fitted transform, but that does not guarantee meaningful placement far from the training data.

Do not say UMAP guarantees global distance preservation. It can provide useful broader structure in some datasets, but distances, density and orientation can distort.

Match each energy task to a reduction goal

PRACTICE

4 min Pairs

Match each energy task to a reduction goal

For each task, name the goal, choose selection or extraction, and name the evidence you would inspect:

1. Plot 25 wells' log suites to spot tool problems before interpretation
2. Store 1 Hz rig data in fewer channels, reconstructible within a tolerance
3. Predict ROP from 60 candidate surface and mud features
4. Decide which of 12 redundant sensors to keep on the next rig

Machine Learning / Module 4 · Dimensionality reduction and feature selection 13

Give 3 minutes in pairs, then 2 minutes of answers.

1. Visualization for quality control, by extraction (a 2-D map). Evidence: neighbourhood preservation (trustworthiness), maps coloured by well and tool after fitting, and every suspected problem confirmed in the original logs.

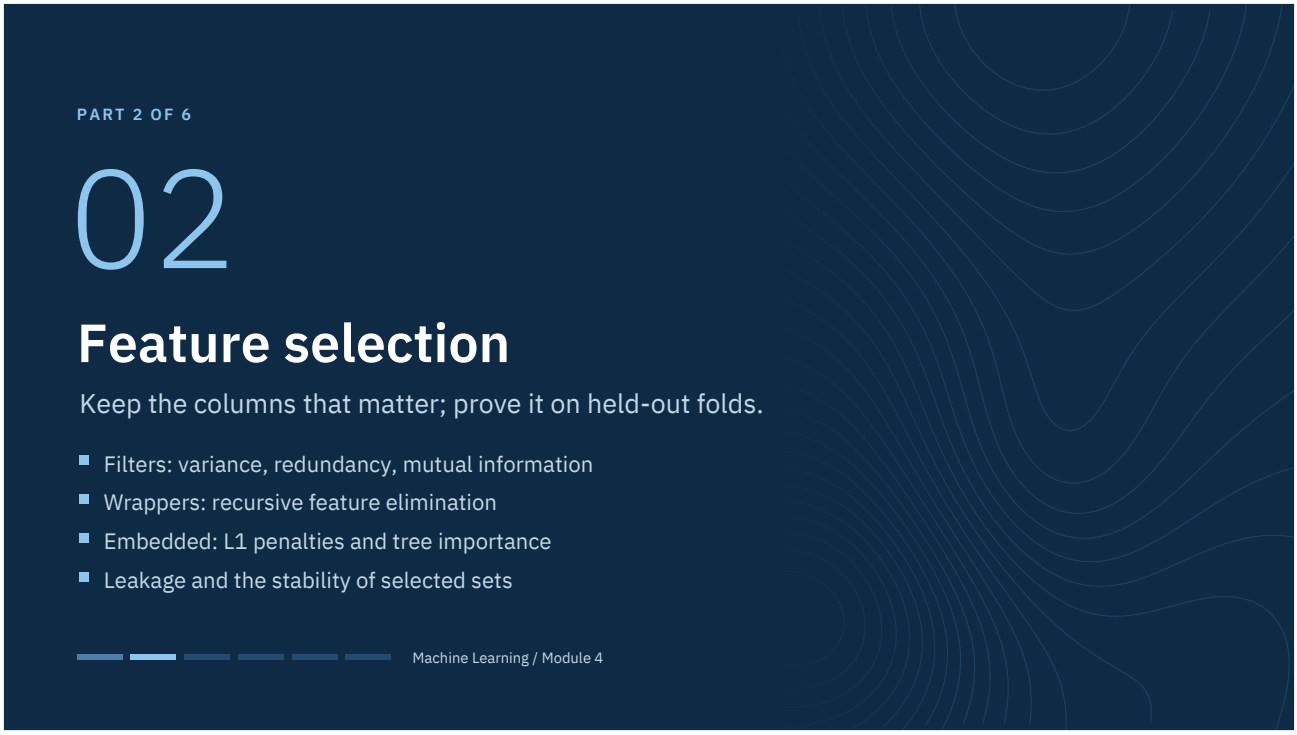
2. Compression, by extraction (PCA). Evidence: reconstruction error per channel in engineering units against the stated tolerance, on a later time period, not just total explained variance.

3. Prediction, by either route, validated. Evidence: held-out score grouped by well, with and without reduction, with the selector or PCA fitted inside each training fold.

4. Prediction or monitoring with fewer sensors, by selection, not extraction. The new rig needs physical sensors, and PCA scores would still require all 12 inputs. Evidence: selection frequency across folds, held-out score of the reduced set, and sensor cost or reliability.

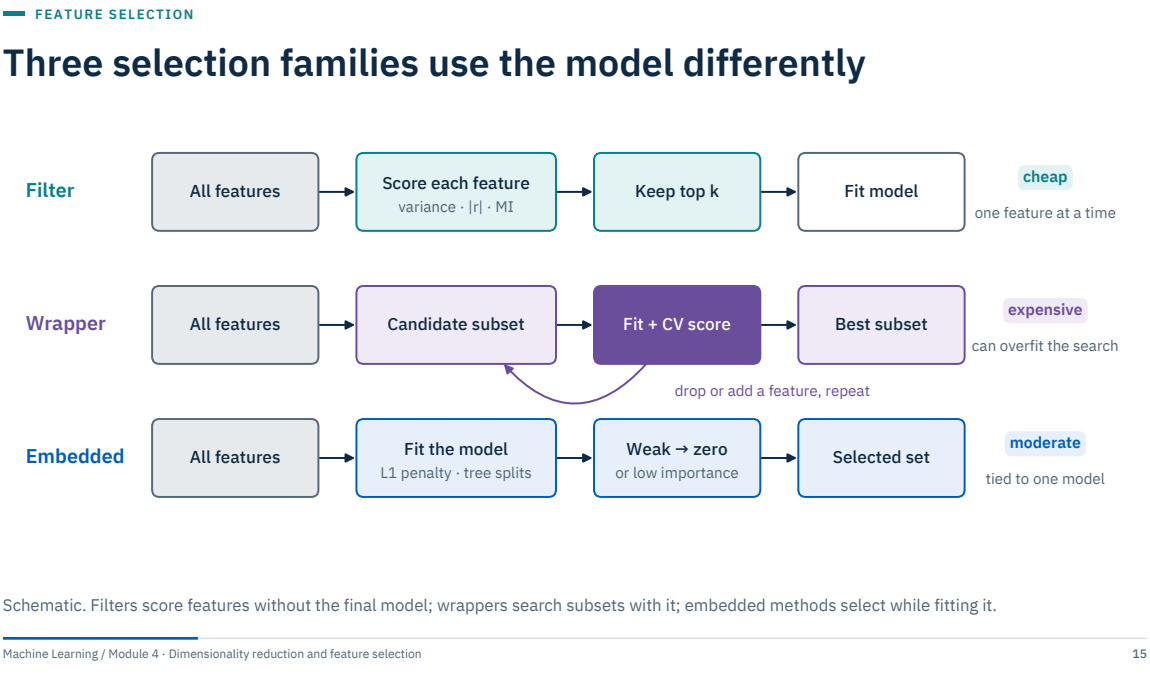
Common error: answering "PCA" for task 4. Another: judging task 3 by explained variance. Transition to Part 2, which covers the selection toolbox.

Feature selection



Part 2 runs about 50 minutes. It reuses tools from Module 1 (lasso, pipelines, grouped cross-validation) and Module 2 (importance, the leakage demonstration) and puts them together as a selection workflow. Two messages dominate: selection is part of the model, so it must happen inside each training fold; and the selected list is itself uncertain, so report how stable it is. The part ends with a leak-hunting exercise. [1 min]

Three selection families use the model differently



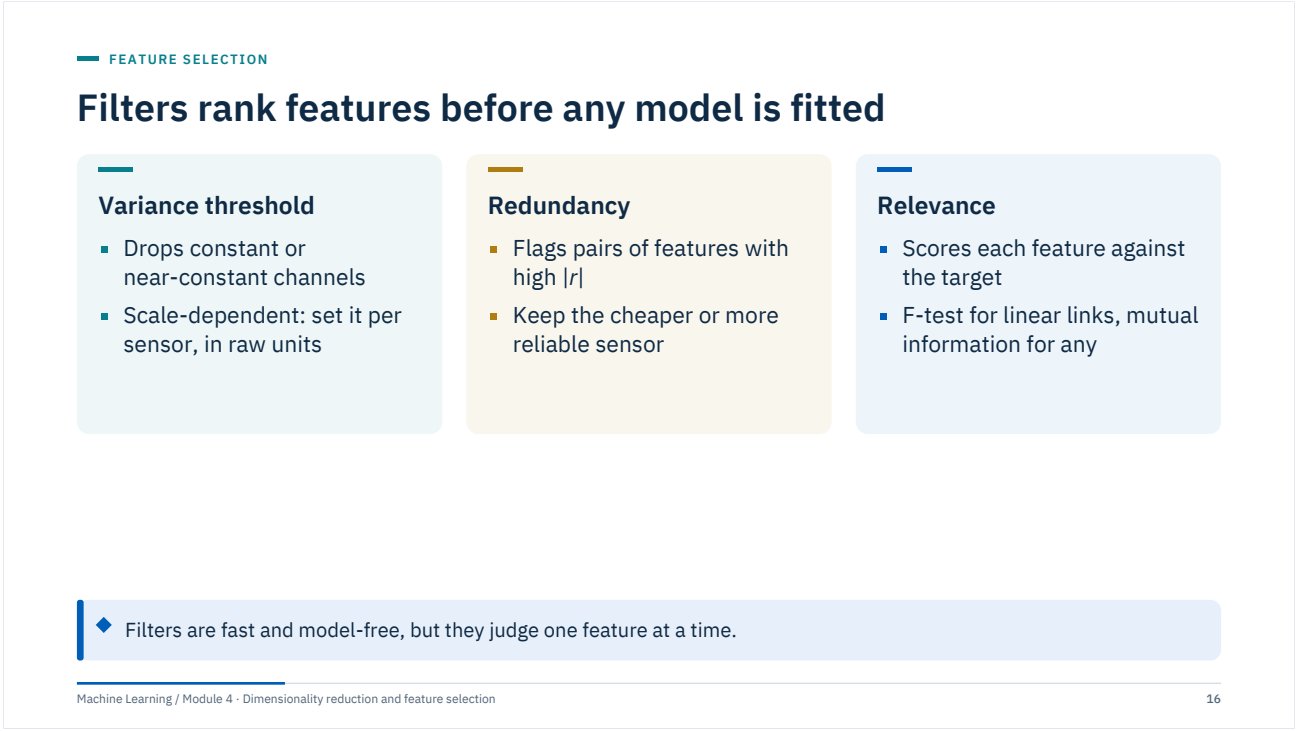
Filters score each feature on its own, with a statistic such as variance, correlation or mutual information, and keep the top k. They are cheap and model-free, but judge one feature at a time, so they can keep three copies of the same signal and miss features that only matter jointly.

Wrappers treat subset choice as a search: propose a subset, fit the model, score it by cross-validation, then add or drop features and repeat. They see interactions through the model but cost many fits, and the search itself can overfit the validation folds.

Embedded methods select as a by-product of fitting: an L1 penalty sets weak coefficients to zero; trees rarely split on weak features. Moderate cost, but tied to one model family.

Ask: which family is your current workflow? Many teams run an informal wrapper by hand: trying subsets and keeping the best score. [3 min]

Filters rank features before any model is fitted



Three kinds of filter [3 min].

Variance threshold (sklearn VarianceThreshold) removes dead or stuck channels, such as a gauge that reads a constant while a rig is idle. It is unit-dependent: a threshold on raw values means different things for psi and ppg, and after standardization every variance is 1, so apply it before scaling and per sensor.

Redundancy filters flag highly correlated pairs; keep the more reliable or cheaper measurement, or summarize the block (next slide).

Relevance filters score each feature against the target: SelectKBest with f_regression or f_classif for linear association, mutual_info_regression or mutual_info_classif for any dependence.

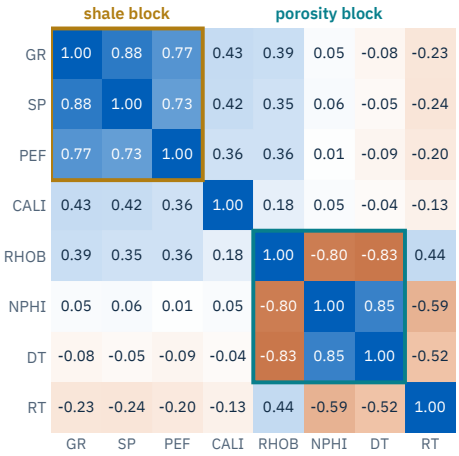
The shared blind spot: a feature that is useless alone but useful in combination (for example a ratio of two measurements) is invisible to univariate scores. Ask for an energy example of such a pair; a classic answer is a pair of pressures whose difference matters.

Correlated logs form redundant blocks

FEATURE SELECTION

Correlated logs form redundant blocks

- GR, SP and PEF: $|r| \geq 0.73$, one shale signal
- RHOB, NPHI and DT: $|r| \geq 0.80$, one porosity signal
- Keep one log per block, or summarize the block with PCA
- Pearson r sees only linear, pairwise links



Synthetic log suite (seed 11); orange cells are negative correlations.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 17

The synthetic suite was generated from shale volume, porosity and water saturation, so the correlation matrix shows two blocks [3 min]. GR, SP and PEF all respond to shale content (pairwise r from 0.73 to 0.88). RHOB, NPHI and DT all respond to porosity (|r| from 0.80 to 0.85; RHOB is negative because density falls as porosity rises). CALI and RT sit apart.

Options for a block: keep the most reliable member (GR works in any mud and through casing, whereas SP needs a water-based mud), average standardized members, or replace the block by its first principal component. Ordering the matrix by hierarchical clustering of 1 – |r| reveals blocks automatically in larger suites.

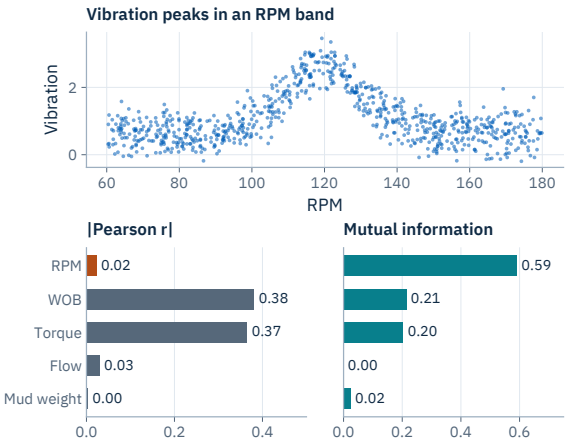
Limits: Pearson r measures linear, pairwise association. Two logs can be strongly but nonlinearly related, and a log can be redundant with a combination of two others without being highly correlated with either. Transition: mutual information addresses the first limit.

Mutual information catches nonlinear relevance

FEATURE SELECTION

Mutual information catches nonlinear relevance

- Vibration peaks in an RPM band, yet |*r*| for RPM is only 0.02
- Mutual information ranks RPM first (0.59): it scores any dependence
- WOB and torque score alike because they carry one signal
- Still one feature at a time: blind to redundancy and interactions



Synthetic example (seed 7): resonance peak plus a weak WOB effect. MI from the k-nearest-neighbour estimator.

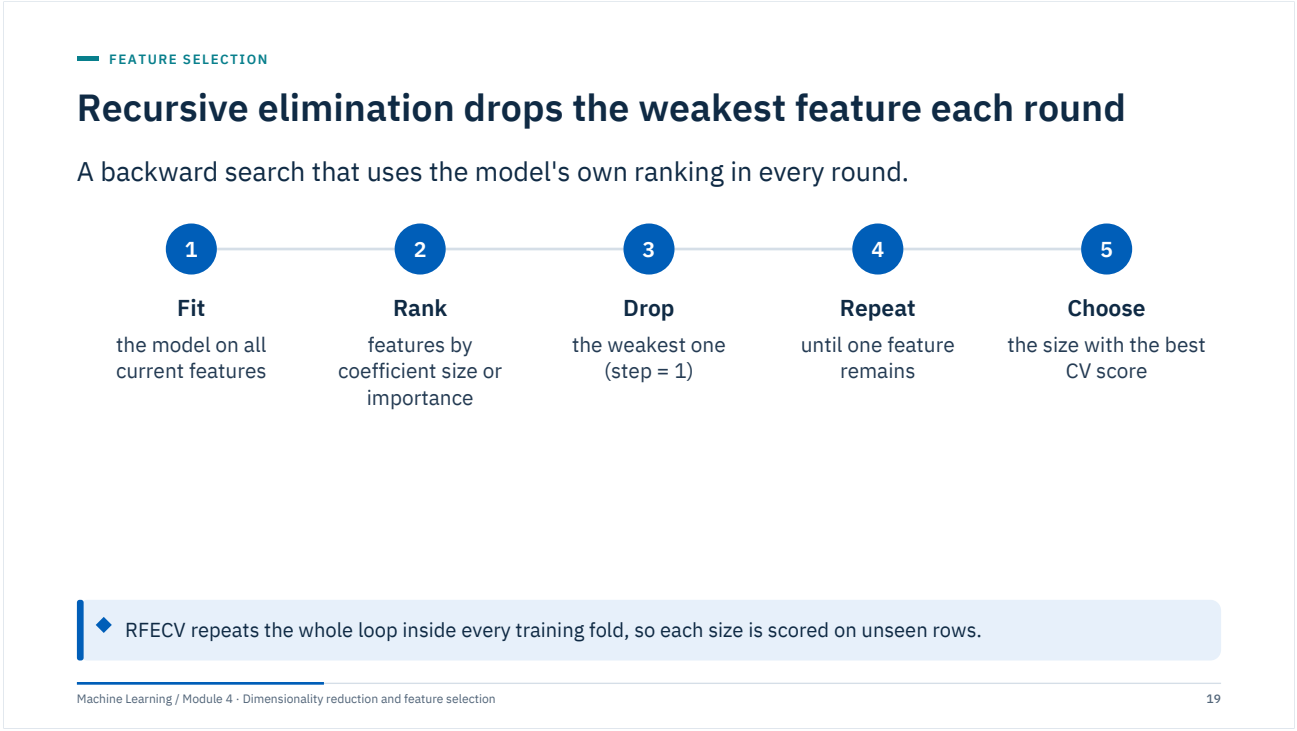
Machine Learning / Module 4 · Dimensionality reduction and feature selection 18

The example mimics a resonance: vibration is high in a band of rotary speeds around 120 rpm and lower on both sides, plus a weak linear effect of WOB [3 min]. Pearson |r| for RPM is 0.02 because the rise and fall cancel; a correlation filter would drop the most important feature. Mutual information is zero only for independent variables, and scores RPM at 0.59, ahead of WOB (0.21) and torque (0.20). Flow and mud weight, which do not enter the target, score near zero.

Practical notes: sklearn estimates MI with a nearest-neighbour method, so values are noisy for small samples; fix random_state and compare features on the same data. MI values are in nats and not bounded by 1, so rank rather than threshold them.

The remaining blind spots: WOB and torque both score about 0.2 because torque is largely driven by WOB; a filter would keep both. Interactions are still invisible.

Recursive elimination drops the weakest feature each round



Recursive feature elimination is the classic wrapper [3 min]. It fits the model, ranks features by the absolute size of the coefficients (linear models, so standardize first) or by feature_importances_ (trees), drops the weakest, and refits. With step = 1 and d features that is about d fits per pass.

RFECV wraps the elimination in cross-validation: for each fold, the elimination runs on the training part and every subset size is scored on the held-out part. The size with the best mean score is chosen, and RFE is refitted on all data at that size.

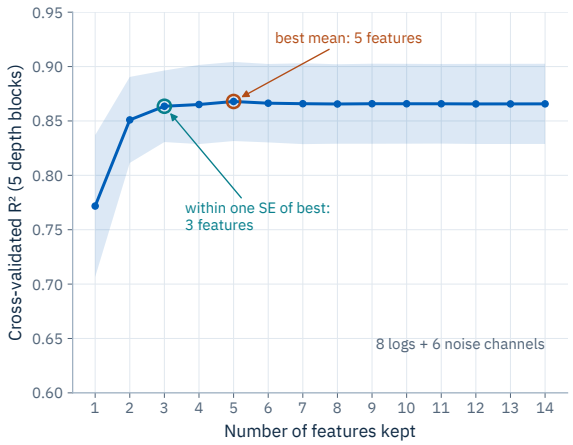
Two cautions. The best mean score is optimistic, because it is the maximum over many sizes; an unbiased estimate needs an outer loop (nested cross-validation, Module 5). And ranking by coefficient size assumes the model and scaling make coefficients comparable. Forward and backward selection (SequentialFeatureSelector) are alternative wrappers.

Cross-validation chooses how many features to keep

FEATURE SELECTION

Cross-validation chooses how many features to keep

- R^2 rises from 0.77 with one log to 0.87 with five, then flattens
- Six noise channels neither help nor clearly hurt this linear model
- Within one standard error of the best: three features
- Folds are contiguous depth blocks, not shuffled rows



RFECV with linear regression; synthetic log suite plus six noise channels; band = ±1 SD across five folds.

The target is log permeability; the candidates are the eight synthetic logs plus six pure-noise channels [3 min]. The curve rises from R^2 = 0.77 with one feature to a best mean of 0.87 with five (GR, SP, RHOB, NPHI, DT), then stays flat. The band is wide (about ±0.04), so sizes from 3 upwards are statistically indistinguishable.

The one-standard-error rule picks the smallest size whose mean is within one standard error of the best (SD / √5 ≈ 0.016): three features, with R^2 = 0.86; refitted on all rows, RFE keeps GR, RHOB and NPHI. That is a defensible choice when every extra log costs money or maintenance.

Folds are contiguous depth blocks (KFold without shuffling) because neighbouring samples in a well are correlated; shuffled folds would put near-duplicates on both sides and inflate the score. With several wells, group by well instead (Module 1). Transition: how the same wrapper looks in code.

Wrapper selectors nest inside an outer cross-validation

FEATURE SELECTION

Wrapper selectors nest inside an outer cross-validation

```
from sklearn.feature_selection import RFECV, SequentialFeatureSelector
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

blocks = KFold(n_splits=5)            # contiguous depth blocks, no shuffle
rfe = RFECV(LinearRegression(), cv=blocks, scoring="r2")
sfs = SequentialFeatureSelector(LinearRegression(), direction="forward",
                                n_features_to_select="auto", tol=0.005,
                                cv=blocks, scoring="r2")
for selector in (rfe, sfs):
    model = make_pipeline(StandardScaler(), selector, LinearRegression())
    outer = cross_val_score(model, X, y, cv=blocks, scoring="r2")
    print(type(selector).__name__, outer.mean().round(3))
```

The inner search runs on each outer training fold only; the outer score is the honest one.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 21

Read the code top to bottom [3 min]. Both selectors live inside a pipeline, so cross_val_score refits scaling and selection on each outer training fold, and the inner cross-validation of the selector runs only on that fold's rows. The outer score is therefore not contaminated by the search.

On the synthetic log suite with six noise channels, the outer R^2 is 0.867 with RFECV, 0.863 with forward selection and 0.866 with no selection at all. Selection did not improve accuracy here; its value is a shorter, cheaper, more interpretable model at no loss. That is a common and honest outcome. Fitted on all rows, forward selection stops at GR, RHOB and NPHI.

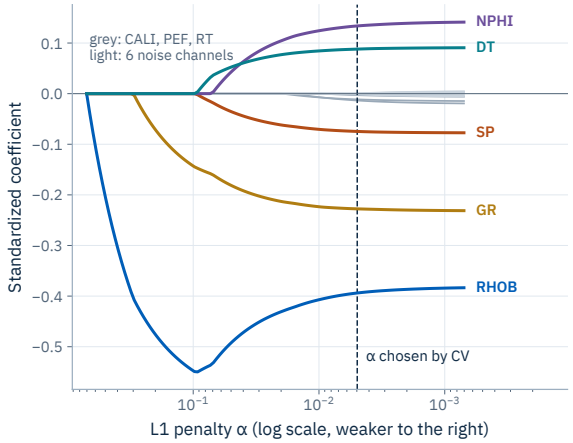
For several wells, use GroupKFold by well for both loops and pass the well labels to the inner selector as well, for example cross_val_score(model, X, y, groups=well_id, cv=GroupKFold(5), params={"rfecv__groups": well_id}); the labels are subset to each training fold automatically.

L1 penalties drive weak coefficients exactly to zero

FEATURE SELECTION

L1 penalties drive weak coefficients exactly to zero

- As α shrinks, features enter: RHOB, then GR, then SP, DT and NPHI
- RHOB's coefficient shrinks once correlated NPHI and DT join
- Standardize first; pick α by CV (dashed line); see Module 1 on the lasso



Lasso path on the synthetic log suite plus six noise channels (target: log permeability).

The lasso adds α times the sum of absolute coefficients to the squared error. Reading the path from left (strong penalty) to right (weak penalty): RHOB enters first, then GR, then SP and DT together, then NPHI; CALI, PEF, RT and the noise channels stay near zero until α is tiny [3 min].

Watch RHOB: its coefficient grows to about −0.55 while it alone carries the porosity signal, then shrinks towards −0.39 as NPHI and DT enter and share that signal. Among correlated features, the lasso's choice can be somewhat arbitrary; elastic net spreads weight across a correlated group.

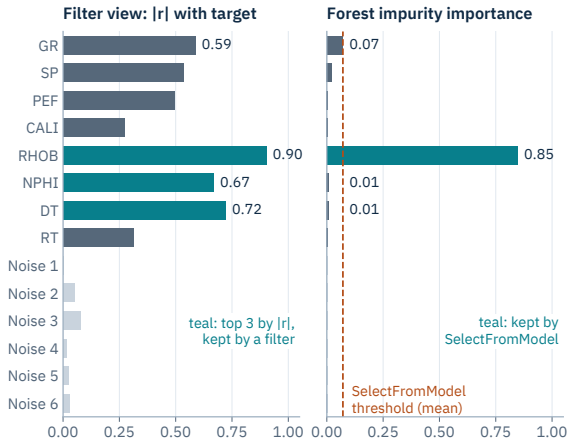
At the α chosen by 5-fold cross-validation (dashed line), two of the six noise channels still have tiny nonzero coefficients (below 0.002 in absolute value). Cross-validation optimizes prediction, not sparsity. For a sparser, more stable set use a larger α (one-standard-error rule) or selection frequencies (two slides ahead). Module 1 covered the lasso as a regression method; here it is a selector.

A forest gives redundant logs almost no importance

FEATURE SELECTION

A forest gives redundant logs almost no importance

- A filter rates NPHI and DT highly: $|r|$ = 0.67 and 0.72 with the target
- The forest credits RHOB with 0.85; NPHI and DT get 0.01, near noise
- SelectFromModel at the mean importance keeps only RHOB
- Redundant is not useless: without RHOB, NPHI and DT carry the signal



Random forest (300 trees, seed 0) on the synthetic log suite plus six noise channels; target log permeability.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 23

Tree ensembles embed selection: a feature that never helps a split gets near-zero importance [3 min]. SelectFromModel turns that into a selector by keeping features above a threshold, here the mean importance (0.071).

Compare the two panels. A filter rates the porosity logs highly: |r| with log permeability is 0.90 for RHOB, 0.72 for DT and 0.67 for NPHI. The forest gives RHOB 0.85 of the impurity importance, and NPHI and DT about 0.01 each, close to the 0.004 of a pure-noise channel. Once the trees split on RHOB, the other porosity logs add almost nothing. So the selector keeps RHOB alone; GR (0.070) misses the threshold by a hair.

This is conditional relevance, not irrelevance. If the density tool failed in a new well, NPHI and DT would carry the porosity signal; a model trained without them could not use it. For robustness, keep one alternative per block or group correlated features before ranking.

Module 2 showed the same sharing with permutation importance and a near-copy of WOB. Transition: the leak that selection creates.

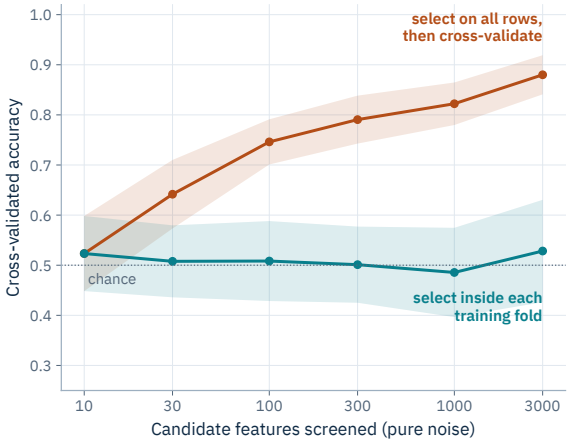
Selection must happen inside cross-validation

FEATURE SELECTION

Selection must happen inside cross-validation

- Pure noise: 60 rows, random labels, keep the 10 best-scoring features
- Selected on all rows: accuracy climbs from 0.52 to 0.88 with more candidates
- Selected inside each training fold: chance at every size

⚠ Any step that looks at the target belongs inside the fold, as in Module 2's leakage demonstration.



30 synthetic pure-noise datasets per point (seeds 2000–2029); SelectKBest (k = 10) and logistic regression; 5-fold CV; bands = ±1 SD.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 24

This is the most important slide of Part 2 [4 min]. Module 2's leakage demonstration showed one setting (50 rows, 2,000 noise features) where selecting on all rows faked a strong signal. Here the same kind of experiment is repeated while the number of screened candidates grows.

Every dataset is pure noise: 60 rows, random labels, so no method can truly beat 50%. Orange: rank all candidates by their F-statistic on all 60 rows, keep 10, then cross-validate a logistic regression on those 10. With 10 candidates SelectKBest keeps them all, so there is no selection and accuracy is 0.52, chance within noise; with 3,000 candidates it reaches 0.88. The more candidates are screened, the more of them match the labels by chance, including on the rows that later act as validation folds.

Teal: the same selection inside a pipeline, refitted on each training fold, stays at chance (0.49 to 0.53) at every size.

Energy relevance: screening hundreds of rig tags, lagged copies or log transforms is exactly the regime where the fake signal is largest. Hastie, Tibshirani and Friedman call this the wrong and right way to do cross-validation (section 7.10.2). Ask: where in your workflows is a ranking done once, up front?

The selector belongs inside the pipeline

FEATURE SELECTION

The selector belongs inside the pipeline

```
from functools import partial
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

mi = partial(mutual_info_classif, random_state=0)
model = make_pipeline(StandardScaler(), SelectKBest(mi, k=10),
                      LogisticRegression(max_iter=1000))
scores = cross_val_score(model, X, y, groups=well_id,
                         cv=GroupKFold(n_splits=5), scoring="f1_macro")
print(scores.mean().round(3), scores.std().round(3))
```

Each fold re-ranks the features on its own training wells; k can be tuned in the same grid search.

The fix for the previous slide is structural: put the selector in the pipeline, so cross_val_score refits scaling, ranking and the classifier on each training fold and scores the held-out wells [3 min]. Module 2 showed this fix with an F-test; this version adds mutual information and grouping by well.

Details worth pointing out. functools.partial fixes random_state for mutual_info_classif, whose nearest-neighbour estimator is otherwise random; the selection becomes reproducible. GroupKFold keeps every row of a well on one side of each split, so the score answers "how well does this work on a new well?". The pipeline also makes deployment consistent: the fitted selector remembers which columns to keep.

To tune k, use GridSearchCV with a grid such as {"selectkbest__k": [5, 10, 20]} on the same pipeline; for an unbiased final estimate, nest that search inside an outer loop (Module 5).

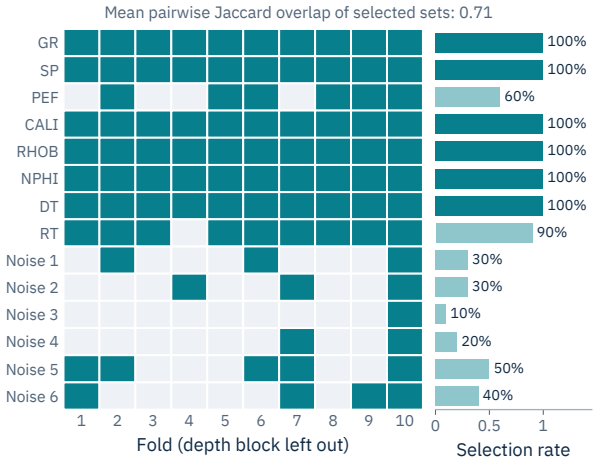
Assumed context: X is a numeric matrix, y class labels, well_id one group label per row. Common error: calling SelectKBest.fit_transform(X, y) before cross_val_score, which reproduces the orange curve.

Selected sets change from fold to fold

FEATURE SELECTION

Selected sets change from fold to fold

- LassoCV refitted in ten depth-block folds
- Six logs chosen in every fold; noise channels in 10–50% of folds
- Mean Jaccard overlap between fold selections: 0.71
- Report selection frequencies, not one "final" list



Synthetic log suite plus six noise channels (seeds 11, 5).

Machine Learning / Module 4 · Dimensionality reduction and feature selection 26

Each column of the grid is one training fold (nine depth blocks), each row a candidate feature; teal means the lasso, with α chosen by its own inner cross-validation, kept that feature [3 min].

Six logs (GR, SP, CALI, RHOB, NPHI, DT) are kept in every fold, RT in 90% and PEF in 60%. The six pure-noise channels are each kept in 10% to 50% of folds, and all of them in fold 10; the size of the selected set ranges from 7 to 14 features. A single run could have reported any of these lists as "the selected features".

The Jaccard index of two sets is the size of their intersection divided by the size of their union; averaged over all pairs of folds it is 0.71 here. A value near 1 means a stable selection.

Practice: report the selection frequency of each feature over folds or bootstrap resamples, and trust features that are selected consistently (the idea behind stability selection). Instability usually signals correlated alternatives or weak features, not a bug. Ask: what would you tell a manager who asks for "the ten features that matter"?

Choose a selection family by cost and blind spot

FEATURE SELECTION

Choose a selection family by cost and blind spot

Family	Cost	Sees interactions	Main risk
Filter	Low	No	Redundant picks; misses joint effects
Wrapper	High	Through the model	Overfitting the search
Embedded	Moderate	Model-specific	Arbitrary picks among correlated features

All three need the same honest validation as the final model.

◆ A robust default: drop dead and duplicate channels first, then an embedded or wrapper method inside grouped CV.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 27

Summarize Part 2 with this table [2 min]. Filters are the first pass for large candidate lists: remove dead channels, collapse obvious redundancy, and rank relevance cheaply. Wrappers suit a modest number of candidates when interactions matter and compute is available. Embedded methods are the default when the final model is a lasso or a tree ensemble anyway.

A common, robust recipe for tabular energy data: drop constant and near-duplicate channels with domain knowledge, then use an embedded or wrapper method inside a grouped cross-validation, and report selection frequencies alongside the held-out score of the chosen set.

Whatever the family, the selector is a fitted component of the model: it goes inside the pipeline, its choices are validated on held-out wells, and its stability is reported. Transition: an exercise to find a leak in a realistic workflow.

Find the leak in this selection workflow

PRACTICE

6 min Pairs

Find the leak in this selection workflow

Thirty wells, 400 candidate drilling features, target ROP. An analyst ranks features by $|r|$ with ROP on all rows, keeps the top 25, then reports a GroupKFold score by well.

1. Where does information leak, and why does grouping by well not prevent it?
2. Restructure the workflow so that the score is honest.
3. What should the report say about the 25 selected features?

Machine Learning / Module 4 · Dimensionality reduction and feature selection 28

Give 4 minutes in pairs, then 2 minutes of answers.

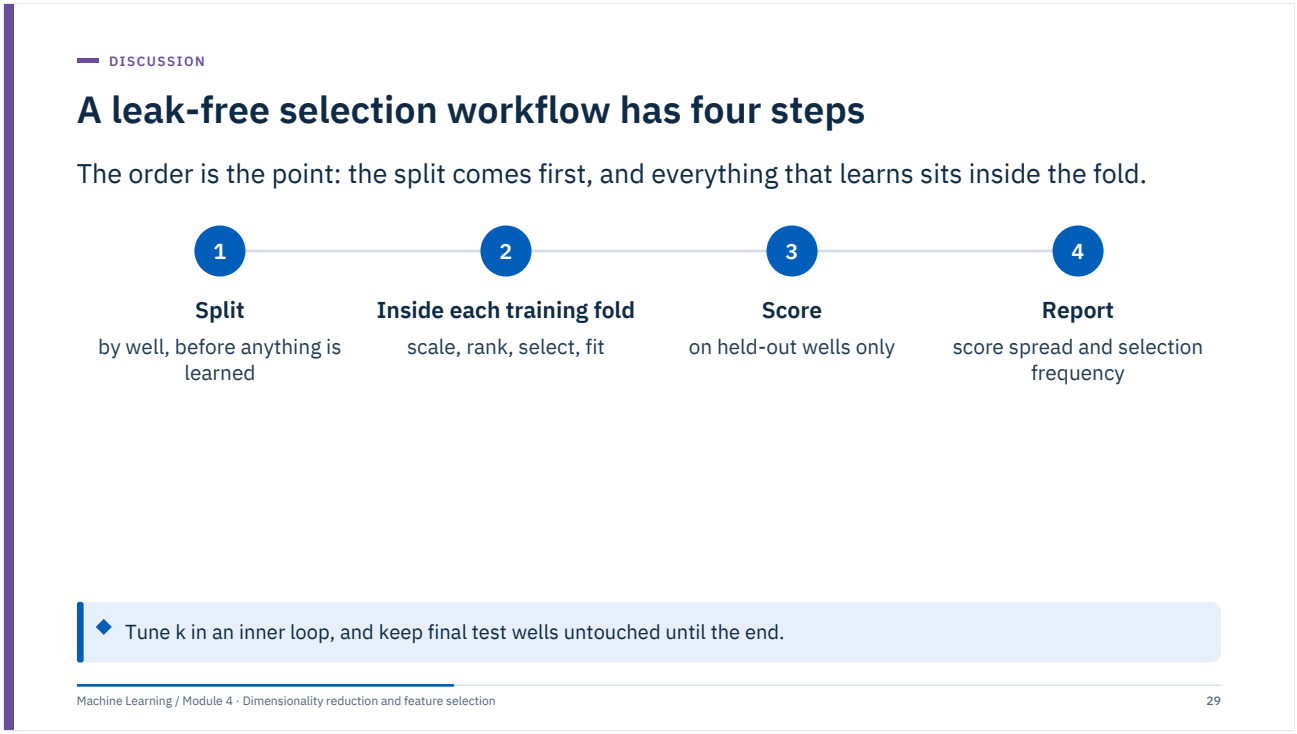
1. The ranking used ROP values from every well, including the wells that later act as validation folds. With 400 candidates, some features correlate with ROP in those wells by chance, and they are carried into every fold. GroupKFold only controls how rows are split for the final model fit; the selection happened before the split, so grouping cannot undo it.

2. Put the ranking inside a pipeline (SelectKBest with f_regression or mutual information, then the model) and cross-validate that pipeline with GroupKFold by well. If k = 25 was chosen by looking at scores, tune it inside an inner loop and keep a final set of test wells untouched until the end.

3. Report selection frequency across folds rather than one list, note groups of correlated features that substitute for each other, and state that the list describes these 30 wells, not causal drivers.

Expect the corrected score to fall; how far cannot be known in advance.

A leak-free selection workflow has four steps



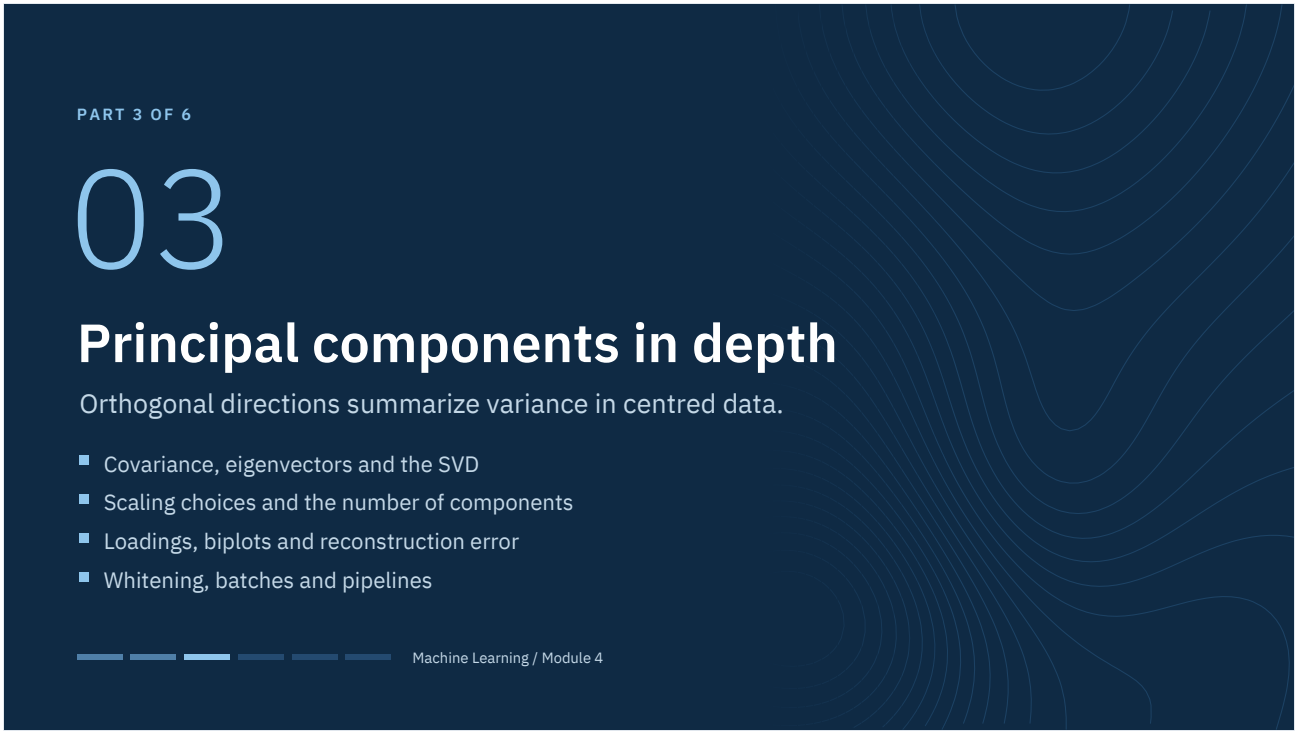
Use this slide to consolidate the exercise answers [2 min]. The order is the whole point: the split comes first, and everything that learns from data, including the selector, sits inside the training fold. The held-out wells are only ever used for scoring.

The report has two parts: the spread of the held-out score across folds (not only its mean), and how often each feature was selected. A feature selected in 10 of 10 folds is a much stronger finding than one selected in 3 of 10.

If k or the selection method is tuned, that tuning is itself fitting: it needs an inner loop inside each outer training fold (nested cross-validation, covered in Module 5). Keep a final set of test wells aside for one last check.

Transition to Part 3: selection keeps original columns; PCA builds new ones. Break first if the clock says 10:30.

Principal components in depth



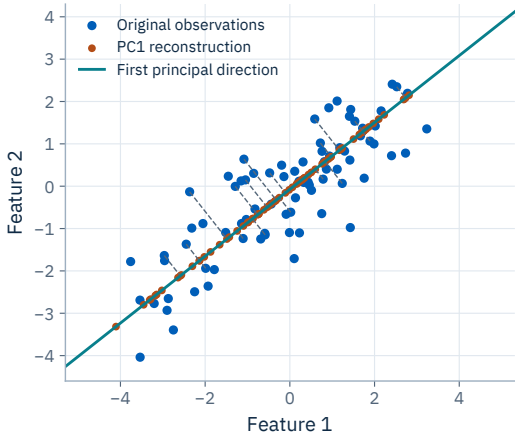
Part 3 runs about 55 minutes and leads straight into Lab 1. Introduce a two-sensor cloud and the first principal direction: ask learners to imagine the line that captures the widest spread when points are projected onto it. Then build up the linear algebra (covariance, eigenvectors, SVD), the practical choices (scaling, number of components), interpretation (loadings, biplots), and uses beyond compression (reconstruction error as an anomaly score). [1 min]

The first principal direction keeps the most spread

PRINCIPAL COMPONENTS IN DEPTH

The first principal direction keeps the most spread

- PC1 is the line along which projected points vary most
- Equivalently, it minimizes squared perpendicular distances (dashed)
- Components are orthogonal, ordered by variance, and sign-arbitrary



PCA computed from a synthetic correlated two-feature dataset. Dashed segments show reconstruction residuals after projection onto PC1.

Blue points are observations; the teal line is the first principal direction; orange points are their projections, which are also the one-component reconstructions [3 min]. Here PC1 retains 92% of the variance.

Two equivalent definitions: PC1 maximizes the variance of the projected points, and it minimizes the sum of squared perpendicular distances (the dashed residuals). Because total variance is fixed, maximizing the kept part minimizes the lost part.

Principal directions are orthogonal and ordered by variance. Reversing the sign of an axis gives the same solution, so never interpret the sign of a single component in isolation.

The first component captures maximum variance, not necessarily the direction most useful for a target. Contrast with regression: least squares minimizes vertical distances to predict y from x; PCA treats both variables symmetrically. Ask: which line would regression of feature 2 on feature 1 draw? (A flatter one.)

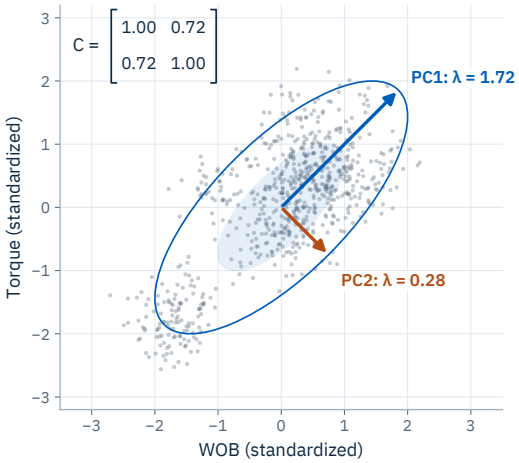
The covariance matrix sets the principal axes

PRINCIPAL COMPONENTS IN DEPTH

The covariance matrix sets the principal axes

$$C v_j = \lambda_j v_j$$

- C: covariance of the centred data, d × d
- Eigenvectors v: the principal directions
- Eigenvalues λ: the variance along each direction
- Standardized WOB and torque: r = 0.72, so λ = 1 ± r = 1.72 and 0.28



Synthetic drilling channels (seed 4); arrows have length 2√λ.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 32

The covariance matrix C = X_c^T X_c / (n − 1) summarizes how every pair of centred features co-varies [3 min]. Its eigenvectors are the principal directions and its eigenvalues are the variances of the data along them. The eigenvalues sum to the total variance (the trace of C).

The figure uses standardized WOB and torque from the motivating case. Their correlation matrix is [[1, 0.72], [0.72, 1]]. For any 2 × 2 correlation matrix the eigenvalues are 1 + r and 1 − r, with directions along the diagonals (1, 1)/√2 and (1, −1)/√2: here 1.72 and 0.28. So PC1 keeps 1.72 / 2 = 86% of the standardized variance.

Ask: what are the eigenvalues if r = 0? (Both 1: no preferred direction, PCA cannot compress.) If r = 1? (2 and 0: one component is exact.) Keep this in mind for Lab 1.

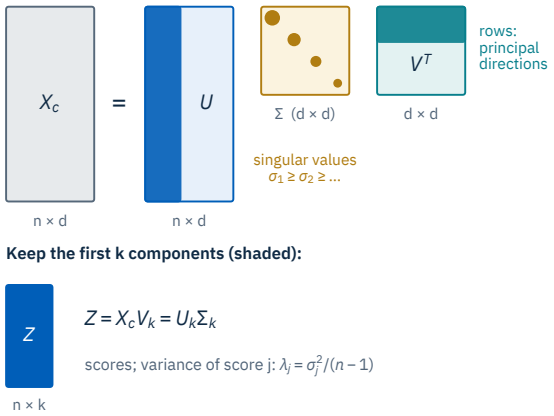
The SVD delivers directions, variances and scores

PRINCIPAL COMPONENTS IN DEPTH

The SVD delivers directions, variances and scores

$$X_c = U\Sigma V^T, \qquad Z = X_c V_k$$

- Centred data: n observations × d features
- Rows of V^T are the principal directions
- Keeping k directions gives the n × k score matrix Z



Schematic of the thin singular value decomposition.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 33

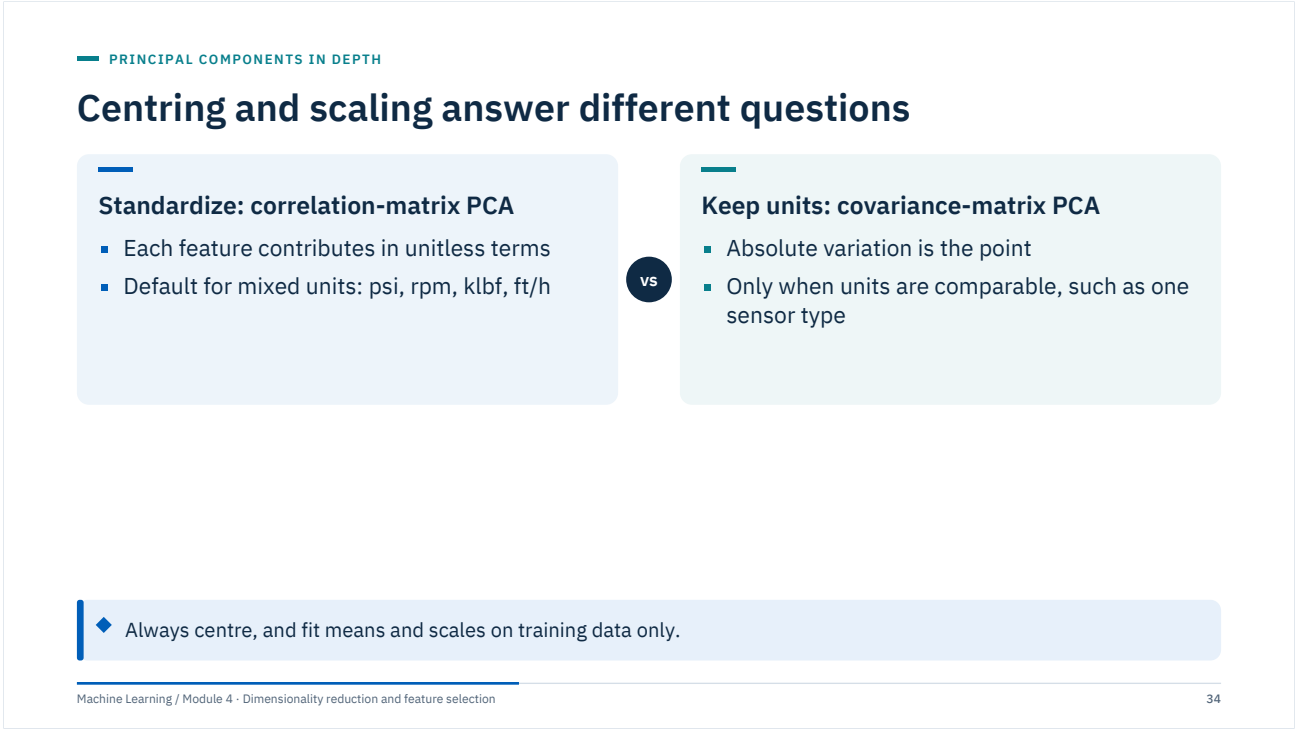
Most implementations, including scikit-learn, compute PCA through the singular value decomposition of the centred data rather than by forming the covariance matrix [3 min]. U holds the left singular vectors (one row per observation), and Σ here is the diagonal matrix of singular values, not a covariance.

The pieces map directly to sklearn attributes: components_ holds the rows of V^T (principal directions); explained_variance_ is σ_j² / (n − 1), which equals the covariance eigenvalues; transform() returns Z = X_c V_k, which also equals U_k Σ_k.

SVD avoids explicitly forming a covariance matrix, which is more accurate numerically and cheaper when d is large. Distinguish components or loading directions (one per feature, d numbers each) from sample scores (one per observation, n numbers each).

For very wide or very large data, svd_solver="randomized" computes only the first k components approximately. Transition: before any of this, a scaling decision.

Centring and scaling answer different questions



PCA centres inputs in scikit-learn but does not standardize their scales automatically [2 min]. Centring is always needed: without it the first component points at the mean rather than along the spread.

Scaling is a modelling decision. Standardizing (subtract the mean, divide by the standard deviation) gives correlation-matrix PCA: every feature starts with variance 1, so a component reflects shared structure rather than units. Keeping units (covariance-matrix PCA) is right when all features share a unit and absolute variation matters, for example a set of identical temperature sensors along a pipeline.

A high-variance pressure sensor can dominate a small-range porosity measurement purely because of units. Inspect distributions and outliers first: one extreme value inflates a standard deviation and shrinks that feature's influence; robust scaling or winsorizing may be justified.

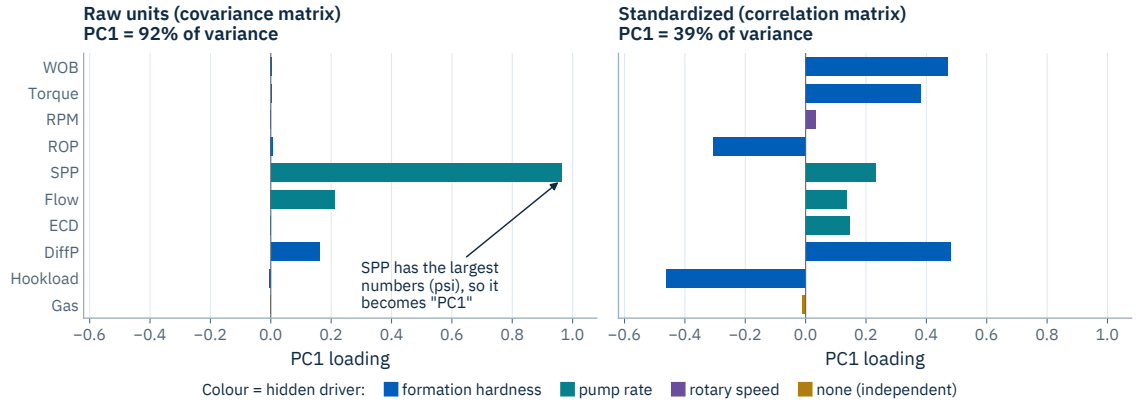
Fit preprocessing only on training data for a predictive workflow. Transition: how much the scaling choice changes the answer.

Standardization changes the answer

PRINCIPAL COMPONENTS IN DEPTH

Standardization changes the answer

Same ten channels, same algorithm: only the scaling differs.



Synthetic drilling channels in engineering units (seed 4). Bar colours mark the driver behind each channel.

Left: PCA on the raw channels. SPP is measured in psi with a standard deviation of about 250; the next largest spread is differential pressure at about 80 psi, and ECD varies by about 0.1 ppg. PC1 is essentially "SPP" (loading 0.96) and claims 92% of the variance. That is a statement about units, not about the rig [3 min].

Right: after standardization, PC1 explains 39% and loads on WOB, torque, differential pressure, hook load (negative) and ROP (negative): the formation-hardness driver from the motivating case. The pump channels appear on PC2.

Ask: which one would you report as "the main mode of variation"? Only the standardized one has a physical reading here. Keep the raw version in mind for Lab 1, where the extension task asks you to compare the two on the course drilling data.

A projection by hand shows what PCA discards

PRACTICE

5 min · Individual

A projection by hand shows what PCA discards

A centred observation is x = [2, 0]. The first direction is v = [1, 1] / √2.

1. Calculate its score on this direction.
2. Reconstruct the observation using this single component.
3. Calculate the squared reconstruction error.
4. What share of the point's squared length did PC1 keep?

Give 3 minutes individually, then work through the answers [5 min].

1. Score $z = x \cdot v = (2 \times 1 + 0 \times 1) / \sqrt{2} = 2 / \sqrt{2} = \sqrt{2} \approx 1.414$.

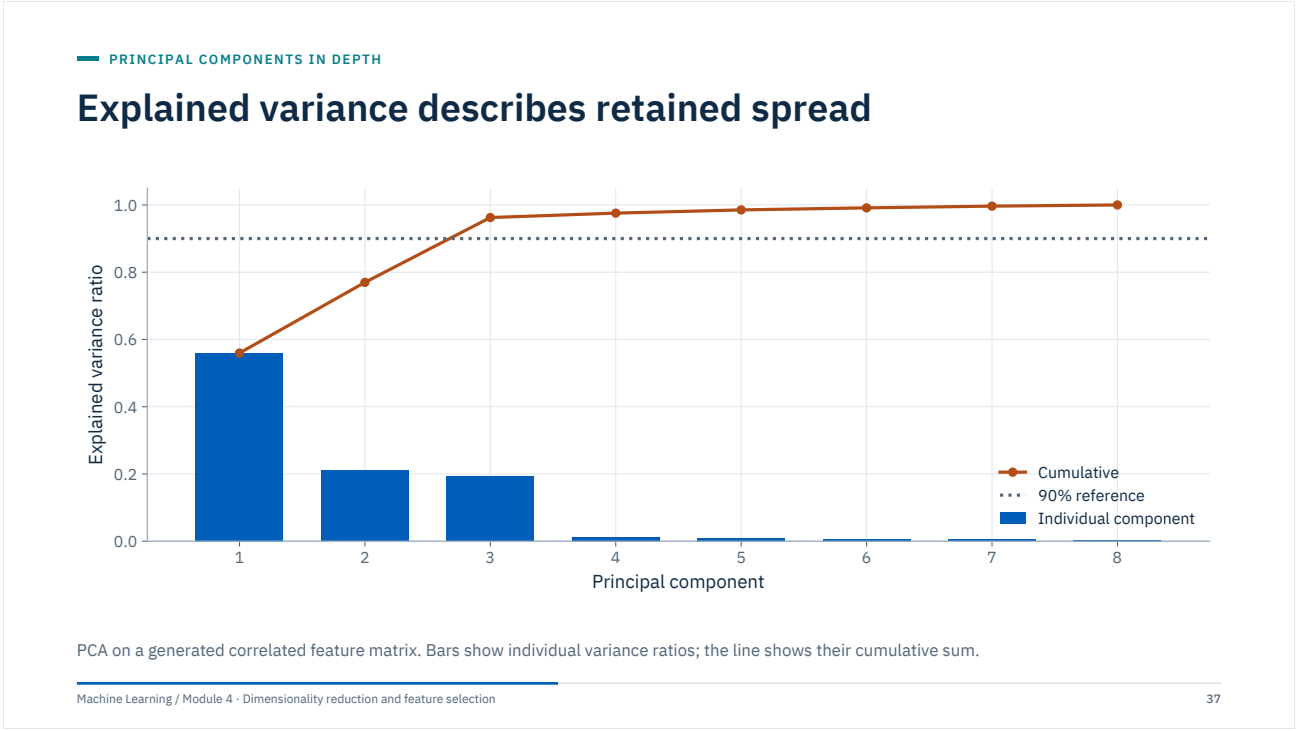
2. Reconstruction $\hat{x} = z\,v = \sqrt{2} \times [1, 1] / \sqrt{2} = [1, 1]$.

3. Residual $x - \hat{x} = [1, -1]$; squared error $= 1^2 + (-1)^2 = 2$.

4. Squared length of x is 4; PC1 kept $z^2 = 2$, that is 50%. Kept plus lost equals the total (2 + 2 = 4), which is Pythagoras: the residual is orthogonal to v.

Emphasize that centring is already done in this example. To reconstruct in original coordinates, add the training mean back: $\hat{x}_{original} = mean + z\,v$. Common errors: forgetting to normalize v to unit length (using [1, 1] directly gives a score of 2 and a wrong reconstruction), and adding the mean before projecting.

Explained variance describes retained spread



This generated matrix has eight features built from three latent factors plus noise [2 min]. The first three components explain 56%, 21% and 19% of the standardized variance, 96% together; components 4 to 8 each explain about 1% or less.

Ask how many components would satisfy a stated 90% threshold in this particular figure. Answer: three (cumulative 77% after two, 96% after three).

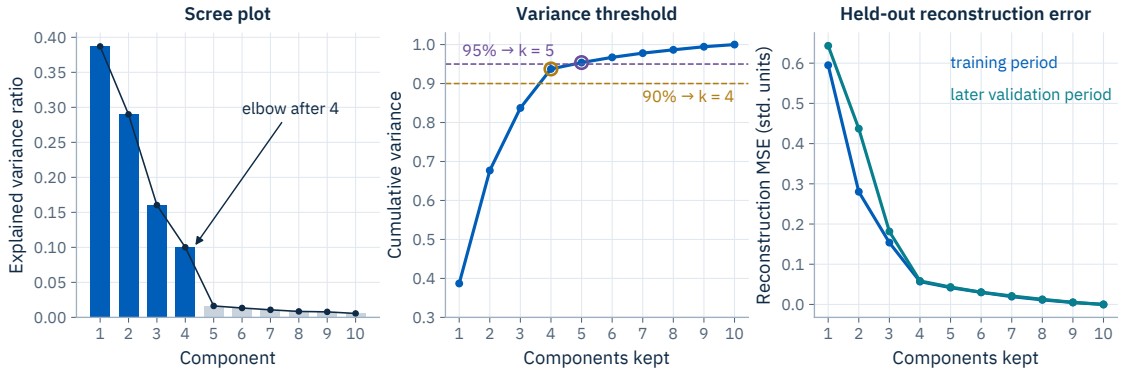
Explain that 90% is a possible compression requirement, not a universal recommendation. Predictive tasks require validation against their target metric, and the component that matters for a target can be one of the small ones. Transition: three ways to choose k, compared on the drilling data.

Three rules for k give related, not identical, answers

PRINCIPAL COMPONENTS IN DEPTH

Three rules for k give related, not identical, answers

Scree elbow after 4; 90% of variance needs 4 components, 95% needs 5; held-out error flattens after 4.



Synthetic drilling channels (seed 4). Left and middle: all 720 minutes. Right: PCA fitted on minutes 0–429, validated on minutes 430–719.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 38

Three common rules applied to the ten drilling channels [4 min].

Scree plot: look for the elbow where the explained-variance ratio drops to a noise floor. Here the drop comes after component 4 (10% for PC4, then 1.6% and below).

Variance threshold: cumulative variance is 38.7%, 67.7%, 83.7%, 93.7% and 95.4% for one to five components, so 90% gives k = 4 and 95% gives k = 5. The threshold is a convention; state it and why.

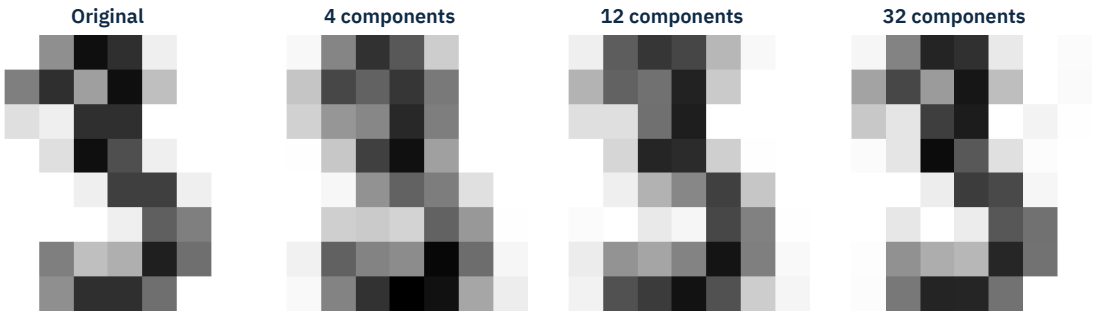
Held-out reconstruction error: fit PCA on the first 430 minutes, reconstruct the later minutes, and plot the mean squared error. The validation error drops steeply to 0.058 at k = 4 and then only slowly. This rule uses data the fit has not seen, so it guards against components that only describe the training period.

Here the rules agree on 4. For prediction the only valid rule is the task score (later in this part).

Reconstruction makes information loss visible

PRINCIPAL COMPONENTS IN DEPTH

Reconstruction makes information loss visible



PCA fitted to the bundled scikit-learn digits (8 × 8 images). Linear compression, not a neural autoencoder.

Each image is an 8 × 8 digit, so 64 features [2 min]. With 4 components the reconstruction is a blurred average shape; with 12 the strokes appear; with 32 it is close to the original.

Ask which details disappear first as components are removed. Answer: fine, rare details such as stroke ends and thin gaps; broad, shared shapes survive because they carry the most variance across the dataset.

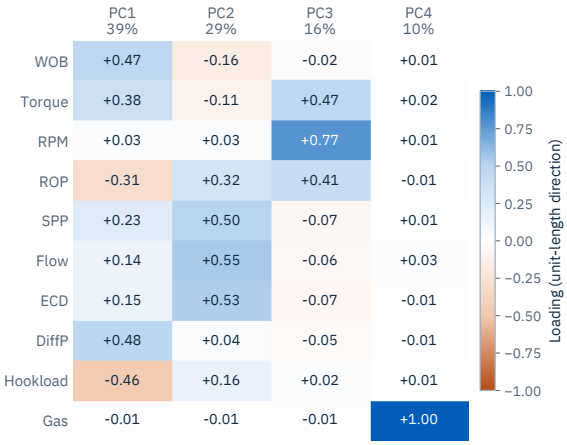
This is a visual teaching example from the bundled digits dataset. For well logs, examine reconstructed curves and errors in original units and by lithology or operating regime. A curve can be reconstructed well on average and badly in exactly the thin beds or events you care about. Transition: which features build each component?

Loadings connect components to features

PRINCIPAL COMPONENTS IN DEPTH

Loadings connect components to features

- Large |*loading*|: the feature drives that direction
- Opposite signs mark a contrast: WOB up, hook load down on PC1
- A component's overall sign can flip without changing the fit
- PC4 is gas alone: a whole component for one channel



Synthetic drilling channels (seed 4), standardized. Rows of components_, signs fixed for display.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 40

Read the heatmap column by column [3 min]. PC1 (39%) loads on WOB, torque and differential pressure (positive) and on hook load and ROP (negative): harder formation, more weight transferred to the bit, slower drilling. PC2 (29%) is the pump group: SPP, flow and ECD. PC3 (16%) is rotary: RPM, torque and ROP. PC4 (10%) is gas alone.

Large absolute coefficients show which features contribute to a direction; opposite signs describe a contrast; the sign of the entire direction can flip without changing the solution. Interpret loadings alongside scaling and correlated predictors.

Clarify terminology: some texts reserve "loading" for eigenvectors scaled by the square roots of the eigenvalues, while sklearn components_ stores unit directions. Label the quantity explicitly in a report.

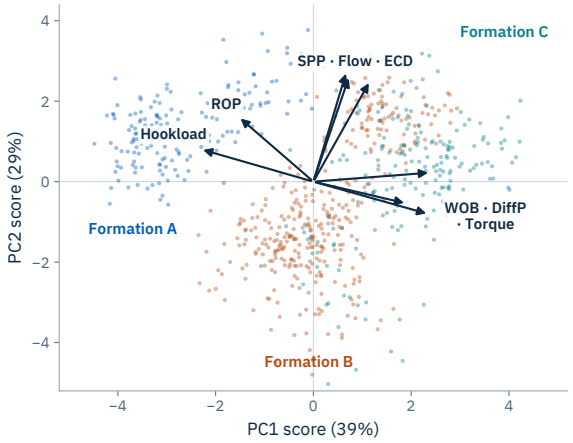
A component mixing sensors is not automatically a physical latent factor. Here we know the generating drivers; in real data, confirm with domain knowledge and events.

A biplot shows observations and loadings together

PRINCIPAL COMPONENTS IN DEPTH

A biplot shows observations and loadings together

- Points: PC1–PC2 scores, coloured by formation after fitting
- Arrows: loadings, rescaled for display
- Formation A sits toward high hook load and ROP, low WOB
- Angles approximate correlations only if two PCs dominate (68% here)



Synthetic drilling channels (seed 4); arrows are loadings × 5 for display. RPM and gas load on PC3–PC4.

A biplot overlays scores (one point per minute) and loading arrows (one per channel) in the same PC1–PC2 plane [3 min].

Reading it: points lying in the direction of an arrow have high values of that channel. Formation A (blue) lies toward the hook-load and ROP arrows and away from WOB and differential pressure: a soft formation drilled with low weight. Formation C lies toward WOB, differential pressure and torque. Vertical spread follows the pump group (SPP, flow, ECD).

Colour was added after fitting; PCA never saw the formation labels. That is how to use metadata: to interpret a representation, not to build it.

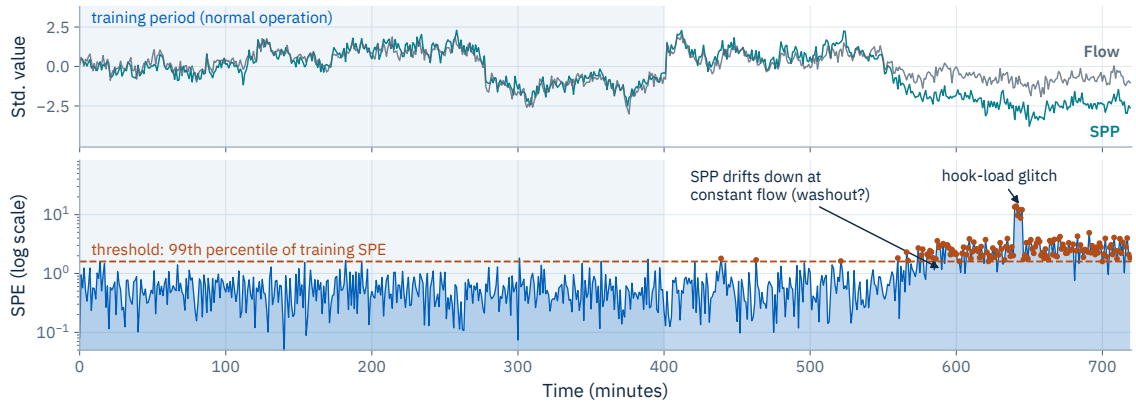
Caveats: arrow lengths depend on a display scale; angles between arrows approximate correlations only when the two plotted components carry most of the variance (68% here, so treat them as rough). RPM and gas have almost no PC1–PC2 loading, so they are invisible here: a biplot can hide channels.

Reconstruction error flags unusual observations

PRINCIPAL COMPONENTS IN DEPTH

Reconstruction error flags unusual observations

Fit PCA on normal operation; score each new minute by its squared prediction error (SPE).



Synthetic drilling channels (seed 4) with an injected SPP drift and a hook-load glitch; PCA with k = 4 fitted on minutes 0–399.

PCA learns which combinations of channels normally move together. A new observation that breaks those relationships reconstructs badly [4 min]. SPE is the squared distance between the standardized observation and its reconstruction from the k components.

Two injected events. From minute 520, SPP falls by up to 450 psi (1.7 training standard deviations) while flow stays constant, a pattern a driller might read as a possible washout. SPE crosses the threshold in 71% of minutes 560–639; a rule requiring 5 alarms in 10 minutes first fires at minute 581. At minute 640, a six-minute hook-load glitch gives SPE of 9 to 14 against a threshold of 1.6.

The threshold is the 99th percentile of training SPE, so about 1% false alarms are expected by construction: here 2 of 120 normal minutes before the drift.

SPE sees broken correlations; unusual values inside the normal pattern (for example very high but consistent WOB) need Hotelling's T^2 on the scores instead. Module 5 compares this detector with isolation forests and one-class methods.

Whitening rescales every component to unit variance

PRINCIPAL COMPONENTS IN DEPTH

Whitening rescales every component to unit variance

$$z_j = \frac{x_c^T v_j}{\sqrt{\lambda_j}}$$

- PCA(`whiten=True`) divides each score by the square root of its eigenvalue
- Whitened scores are uncorrelated and have equal variance
- A standard first step for ICA and some distance-based models
- It inflates noise in small components, so keep k modest

Machine Learning / Module 4 · Dimensionality reduction and feature selection 43

Whitening is a small extension [2 min]. Ordinary PCA scores are uncorrelated but have variances equal to the eigenvalues λ_j; whitening divides each score by √λ_j so that every component has variance 1. The point cloud becomes spherical.

Why do it: some methods assume or benefit from equal-variance, uncorrelated inputs. FastICA whitens internally before looking for independent directions; distance-based methods then treat every retained component equally.

The risk: a component with a tiny eigenvalue is often mostly noise, and whitening multiplies it up to the same weight as PC1. Always combine whitening with a modest number of components chosen by validation.

In scikit-learn, whiten=True changes transform() output only; components_ are unchanged, and inverse_transform() undoes the scaling. Ask: why would whitening with all components make k-means distances meaningless? (Noise dimensions get the same weight as signal.)

Incremental PCA fits data in batches

PRINCIPAL COMPONENTS IN DEPTH

Incremental PCA fits data in batches

```python
import pandas as pd
from sklearn.decomposition import IncrementalPCA
from sklearn.preprocessing import StandardScaler

cols = ["WOB", "TORQUE", "RPM", "ROP", "SPP", "FLOW"]
scaler = StandardScaler()
for chunk in pd.read_csv("rig_1hz.csv", usecols=cols, chunksize=100_000):
    scaler.partial_fit(chunk)                    # pass 1: means and SDs

ipca = IncrementalPCA(n_components=4)
for chunk in pd.read_csv("rig_1hz.csv", usecols=cols, chunksize=100_000):
    ipca.partial_fit(scaler.transform(chunk))    # pass 2: components
print(ipca.explained_variance_ratio_.round(3))
```

Useful when a year of 1 Hz rig data does not fit in memory; check against batch PCA on a sample.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 44

The file name is a placeholder for any large table [2 min]. IncrementalPCA updates its components one batch at a time, so memory depends on the batch size, not on the number of rows. Two passes are needed: the scaler must see all data before the components are fitted, and StandardScaler also supports partial_fit.

Each batch must contain at least n_components rows. The result approximates batch PCA; with large batches the difference is usually negligible, but verify once on a sample that fits in memory. In a test on 250,000 synthetic rows, the incremental and batch explained-variance ratios agreed to three decimals.

Alternatives: svd_solver="randomized" for wide data that fits in memory, or subsampling when the goal is exploratory. For a model that must learn continuously, remember that a changing rig or formation also changes the components; refitting is a monitoring decision (Module 5). Transition: PCA inside a predictive pipeline.

PCA belongs inside the prediction pipeline

PRINCIPAL COMPONENTS IN DEPTH

PCA belongs inside the prediction pipeline

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, GroupKFold

model = make_pipeline(StandardScaler(), PCA(),
                      LogisticRegression(max_iter=1000))
grid = {"pca__n_components": [2, 4, 6, 8, 10]}
search = GridSearchCV(model, grid, cv=GroupKFold(n_splits=5),
                      scoring="f1_macro")
search.fit(X_train, y_train, groups=wells_train)
print(search.best_params_, round(search.best_score_, 3))
```

Compare with the same pipeline without PCA, on the same folds.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 45

This snippet assumes complete numeric predictors (at least ten, so every grid value is valid) and one well label per row; add imputation where necessary [2 min].

The pipeline refits the scaler and the PCA on each training fold, so the validation wells never influence the components. n_components is tuned like any hyperparameter, by the task score on held-out wells. Never fit PCA on the combined training and test data for this predictive use case: that leaks the test distribution into the features.

best_score_ is the best mean across the grid and therefore slightly optimistic; report a final score on untouched test wells or use nested cross-validation (Module 5).

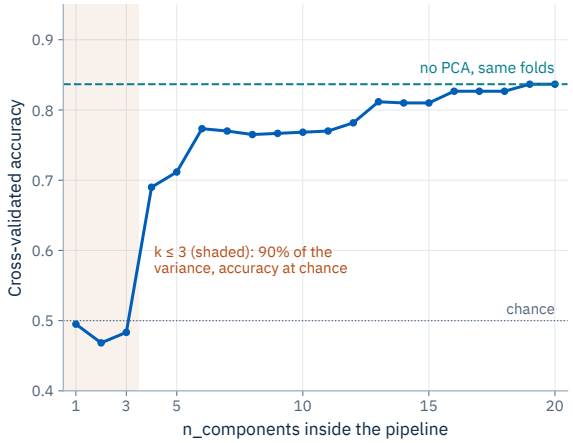
Most important: run the same search without the PCA step on the same folds. If PCA does not beat or match that baseline, it has not earned its place in the model. Transition: an example where it clearly does not.

Tune the number of components against the task

PRINCIPAL COMPONENTS IN DEPTH

Tune the number of components against the task

- Three nuisance factors hold 90% of the variance and no class signal
- With k ≤ 3 the classifier sits at chance (0.47–0.50)
- It matches the no-PCA baseline (0.84) only at k = 19



Synthetic 20-feature data (seed 3); pipeline: StandardScaler, PCA, logistic regression; 5-fold CV.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 46

A constructed but realistic trap [3 min]. Twenty features are driven by three strong nuisance factors, think operating regimes, which hold 90% of the variance. The class signal, think an incipient fault, lies in a direction with little variance.

Keeping the top three components, the choice any variance rule would make, gives accuracy 0.47 to 0.50: chance. Accuracy jumps once component 4 or later enters, and only reaches the no-PCA baseline of 0.84 at k = 19 of 20.

The general lesson: PCA is unsupervised. It ranks directions by variance, and the target may live in a small one. Before putting PCA in front of a model, compare with the baseline on the same folds, and treat k as a hyperparameter. If you need a supervised reduction, consider feature selection (Part 2), partial least squares, or LDA (Part 4). Transition: the classic exercise on this point.

PCA limitations are testable

PRACTICE

5 min Pairs

PCA limitations are testable

Pressure explains most variance, but a small vibration feature predicts failures.

1. Would keeping only high-variance components necessarily help classification?
2. What experiment would test this?
3. Which method from Part 2 could keep the vibration feature?

Give 3 minutes, then discuss [5 min].

1. No. PCA ranks directions by variance, not by relevance to failures. If the vibration feature has small variance and little correlation with pressure, it will sit in a late component and be discarded by a variance threshold. After standardization it has the same variance as pressure, but it may still be outranked by a block of correlated pressure channels.

2. Fit PCA only inside the training folds and compare a baseline classifier with and without PCA on the same held-out metric and the same grouped folds (for example by pump or well). Tune n_components as a hyperparameter. Look at the loading of vibration on the retained components.

3. Supervised feature selection: a filter with mutual information, an L1-penalized logistic regression, or RFECV would evaluate vibration by its relation to the failure label. Consider retaining more components too.

Variance and class discrimination are different objectives. Transition to Lab 1.

Lab 1: PCA on drilling data

LABORATORY

60 min

Lab 1: PCA on drilling data

- ☐ Check the correlation matrix first; treat Depth as an index, not a feature
- ☐ Standardize ROP, RPM, Torque and WOB; fit PCA with all components
- ☐ Choose k from scree and cumulative plots, and justify the threshold
- ☐ Read loadings and the biplot; report reconstruction error in original units
- ☐ Extension: repeat PCA on raw units and compare PC1 with the standardized fit

NOTEBOOK

`exercises/01_pca/PCA_Drilling_Student.ipynb`

EVIDENCE TO SUBMIT

- Scree and cumulative-variance plots with your chosen k
- A loadings table with one sentence per component
- Reconstruction RMSE per feature in original units

Machine Learning / Module 4 · Dimensionality reduction and feature selection 48

Timing: 60 minutes (11:40–12:40), debrief after lunch (13:40). The solution is exercises/01_pca/PCA_Drilling_Solution.ipynb. The dataset has 100 rows (depth 1,000 to 2,000, step about 10) and four drilling features. The notebook draws its heatmaps with seaborn, which is in requirements-labs.txt. Every fit uses all 100 rows and is labelled exploratory. Section 7 reports reconstruction RMSE per feature in original units for one to four components; section 8 (PCA before k-means) links to Module 3 and also scores the PCA-space clusters in the standardized space; section 9 is the raw-unit extension.

Circulate with these prompts. Before fitting: "What does the correlation matrix predict about compression?" After the scree plot: "Is there an elbow?" For the loadings: "Can you name each component in drilling terms?"

Common errors: including Depth as a feature (it is an index); forgetting to standardize (Torque and WOB then dominate); computing reconstruction error in standardized units only; reading a 95% threshold as mandatory. Fast finishers: the raw-unit extension, then k-means on PC scores versus on standardized features, comparing silhouette scores.

Do not reveal the result before the debrief: the answer is surprising and worth discovering.

Lab 1 debrief: this drilling table barely compresses

DISCUSSION

Lab 1 debrief: this drilling table barely compresses

The course file drilling.csv, analysed as in the notebook:

28.5%	4 of 4	89%
PC1 share after standardizing	components needed for 90% of variance	PC1 share in raw units: WOB alone

◆ A flat spectrum is a finding: the four channels are nearly uncorrelated ($|r| \leq 0.13$), so there is little to compress.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 49

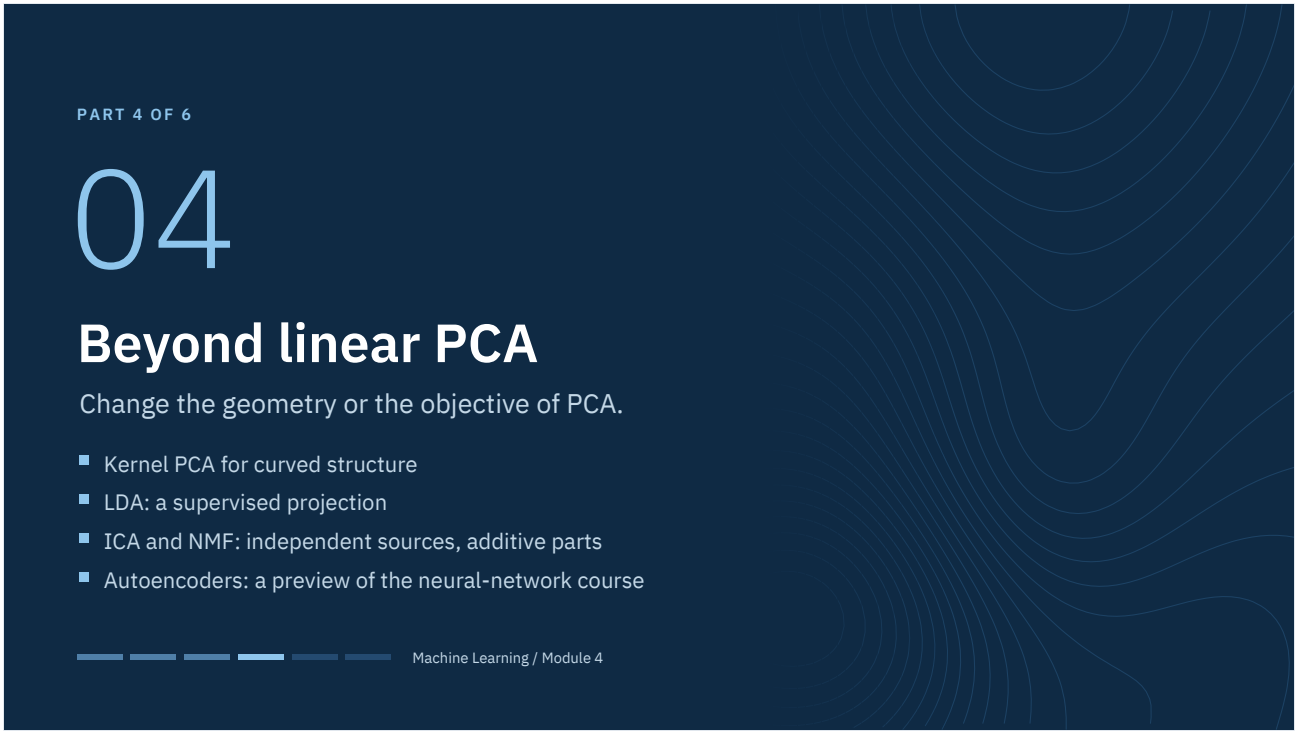
Values computed from drilling.csv [15 min including discussion]. After standardizing, the explained-variance ratios are 28.5%, 27.6%, 24.3% and 19.6%; the cumulative share reaches 80.4% only at three components, so both the 90% and the 95% thresholds require all four. The correlations between the four features are all below 0.13 in absolute value.

Ask groups what they concluded. Good answers: "PCA finds no dominant structure; each channel carries its own information; reduction is not justified for compression." The notebook's automatic 95% rule returns k = 4, which is correct and informative, not a failure.

Groups that skipped standardization saw PC1 claim 89% of the variance with a loading of 0.999 on WOB, because WOB has the largest numeric spread (standard deviation about 4,300 versus about 1,500 for torque). That was a statement about units.

Reconstruction: with three standardized components the RMSE is about 1.2 for ROP but about 1,800 for WOB, each in the file's own units (the CSV does not state them); a real compression tolerance would reject that. Transition to Part 4.

Beyond linear PCA



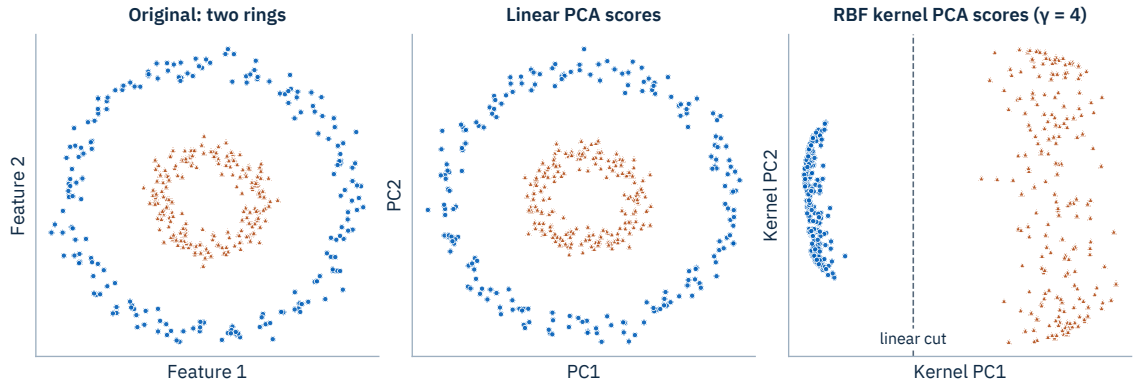
Part 4 runs about 30 minutes after the Lab 1 debrief. Each method keeps the idea of a low-dimensional representation but changes one ingredient of PCA: kernel PCA changes the geometry through a similarity function; LDA changes the objective to class separation using labels; ICA changes it to statistical independence; NMF adds a non-negativity constraint; autoencoders replace the linear map with a neural network. The part ends with a method-choice exercise. [1 min]

Kernel PCA can unfold what linear PCA cannot

BEYOND LINEAR PCA

Kernel PCA can unfold what linear PCA cannot

PCA runs on an n × n similarity (kernel) matrix instead of the d × d covariance matrix.



scikit-learn make_circles (400 points, seed 0); RBF kernel, γ = 4. Colours show ring membership, which is never used in fitting.

Two concentric rings [3 min]. Linear PCA can only rotate and rescale, so its scores (middle) are the same rings: no linear direction separates inner from outer.

Kernel PCA replaces dot products between observations with a similarity $k(x, x')$; with the RBF kernel, $k = \exp(-\gamma\|x - x'\|^2)$. It then performs PCA on the centred n × n kernel matrix, which is equivalent to linear PCA in an implicit, very high-dimensional feature space. On the right, the first kernel component places the whole inner ring at one end and the outer ring at the other, so a single threshold on kernel PC1 separates them.

The labels were used only for colouring, and $\gamma = 4$ was chosen by eye for this illustration. In practice γ must be tuned, ideally by a downstream score, and a badly chosen γ gives nonsense. Transition: what this flexibility costs.

Kernel PCA trades interpretability for flexibility

BEYOND LINEAR PCA

Kernel PCA trades interpretability for flexibility

	Linear PCA	Kernel PCA
Works on	d × d covariance	n × n kernel matrix
Captures	Linear directions	Curved structure, via the kernel
Memory and time	Modest for large n	Grows with n^2 and n^3
Interpretation	Loadings per feature	No loadings; approximate pre-images
New observations	Exact projection	`transform()` via kernels to training rows

◆ Tune γ against a downstream score; without one, prefer linear PCA.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 52

Walk through the rows [3 min]. The kernel matrix has one row and column per training observation, so memory grows with n^2 and an exact eigendecomposition with n^3. Ten thousand log samples mean a 10^8-entry matrix; subsample, or use eigen_solver="randomized" or approximate kernel features (Nystroem) for large data.

There are no loadings, because the components live in the implicit feature space, not in the original features. Reconstruction in original units needs an approximate pre-image: KernelPCA(fit_inverse_transform=True) learns one by kernel ridge regression.

New observations are projected by computing their kernel values against all training rows, so the training set is part of the fitted model and must be stored.

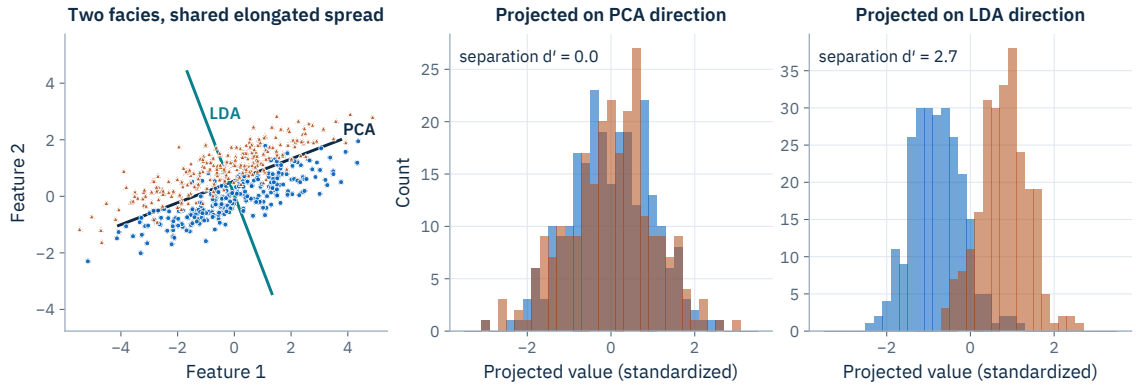
Rule of thumb: try kernel PCA when there is evidence of curved structure and a downstream score to tune γ against; otherwise prefer PCA or a model that handles nonlinearity directly. Transition: a supervised projection.

LDA projects to separate labelled classes

BEYOND LINEAR PCA

LDA projects to separate labelled classes

PCA follows the largest spread; LDA follows the direction that separates the labelled classes.



Synthetic two-class data with a shared covariance (seed 2). d′ = difference in class means / pooled standard deviation.

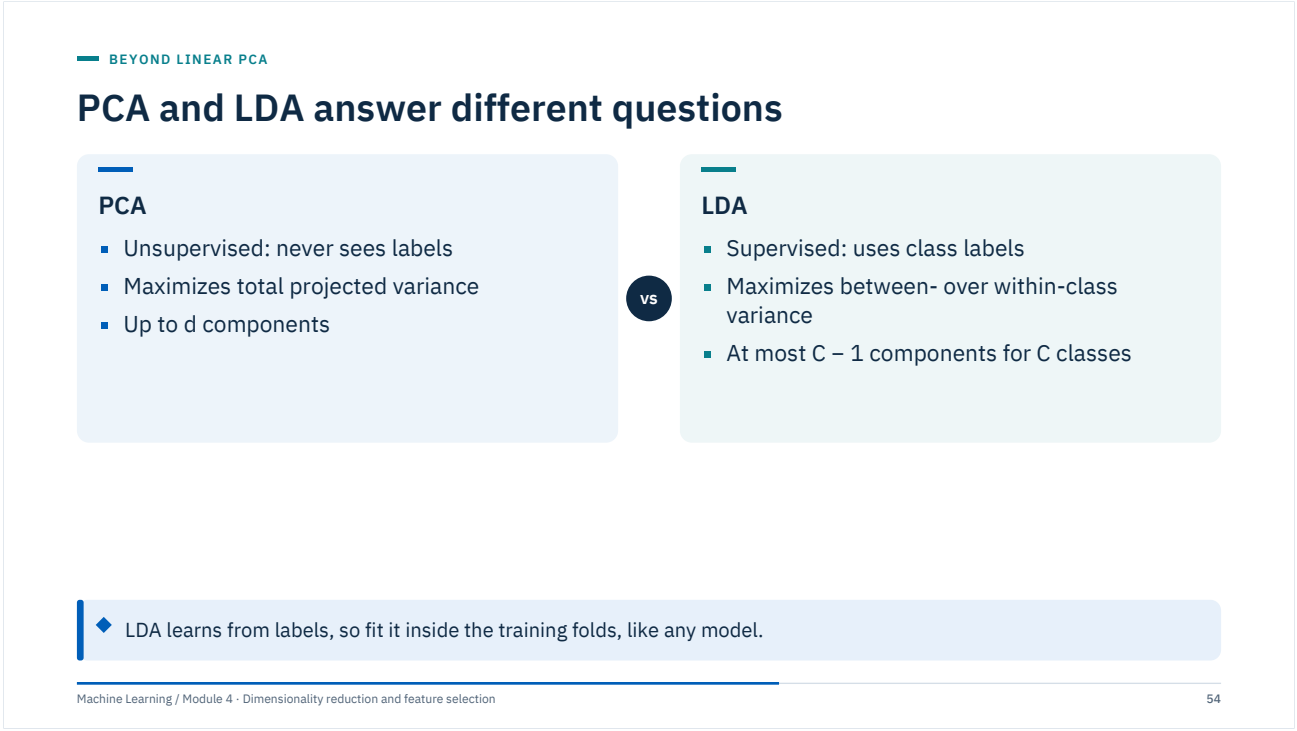
Machine Learning / Module 4 · Dimensionality reduction and feature selection 53

Two facies share the same elongated spread and differ by a small shift across it [3 min]. PCA picks the long axis, because that is where the variance is; projected on it, the two classes overlap completely (d′ = 0.0). Linear discriminant analysis uses the labels and picks the direction that maximizes the ratio of between-class to within-class variance; projected on it, the classes separate (d′ = 2.7).

d′ is a simple separation measure: the difference of class means divided by the pooled standard deviation. Values near 0 mean complete overlap.

This is the supervised counterpart of the PCA trap from Part 3: when the discriminating direction has little variance, an unsupervised method discards it. LDA assumes roughly Gaussian classes with a shared covariance; it still works as a projection when these hold loosely, but check held-out performance. Transition: side-by-side summary.

PCA and LDA answer different questions



Key contrasts [2 min]. PCA can be fitted without labels and produces up to d components ordered by variance. LDA needs labels and produces at most C – 1 components, where C is the number of classes: two for three facies, one for a binary problem.

Because LDA uses labels, a map built with it is evidence only about the data used to fit it. Colouring an LDA map by the labels that shaped it and concluding that "classes are well separated" is circular; fit on training wells, then project and judge held-out wells.

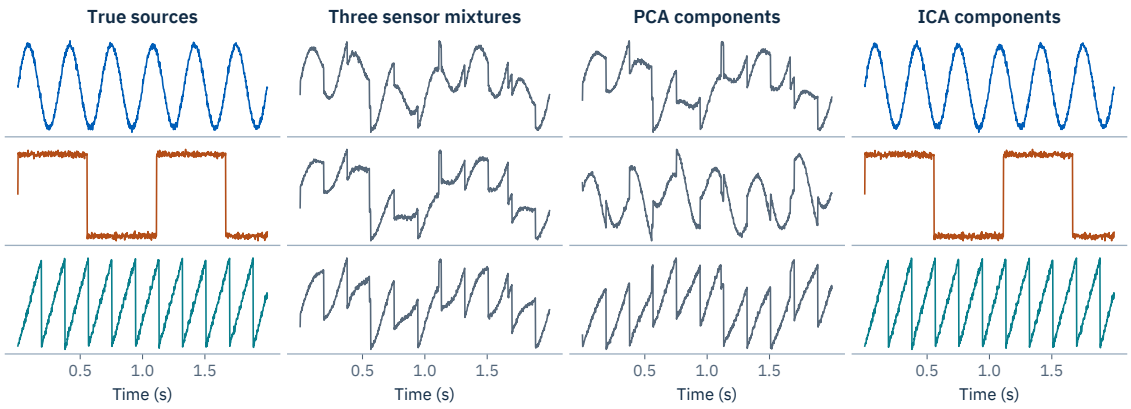
LinearDiscriminantAnalysis in scikit-learn is both a classifier and a transformer (transform() gives the discriminant scores). With many correlated features and few rows, use shrinkage: solver="eigen" with shrinkage="auto" (the "lsqr" solver also shrinks but has no transform).

Ask: you have three facies and 12 logs; how many LDA components can you plot? (Two.) Transition: a different objective altogether.

ICA unmixes sources that PCA only decorrelates

BEYOND LINEAR PCA

ICA unmixes sources that PCA only decorrelates



Synthetic sources mixed by a fixed 3 × 3 matrix; FastICA (seed 0). ICA rows are reordered and sign-matched for display, because ICA cannot recover order, sign or scale.

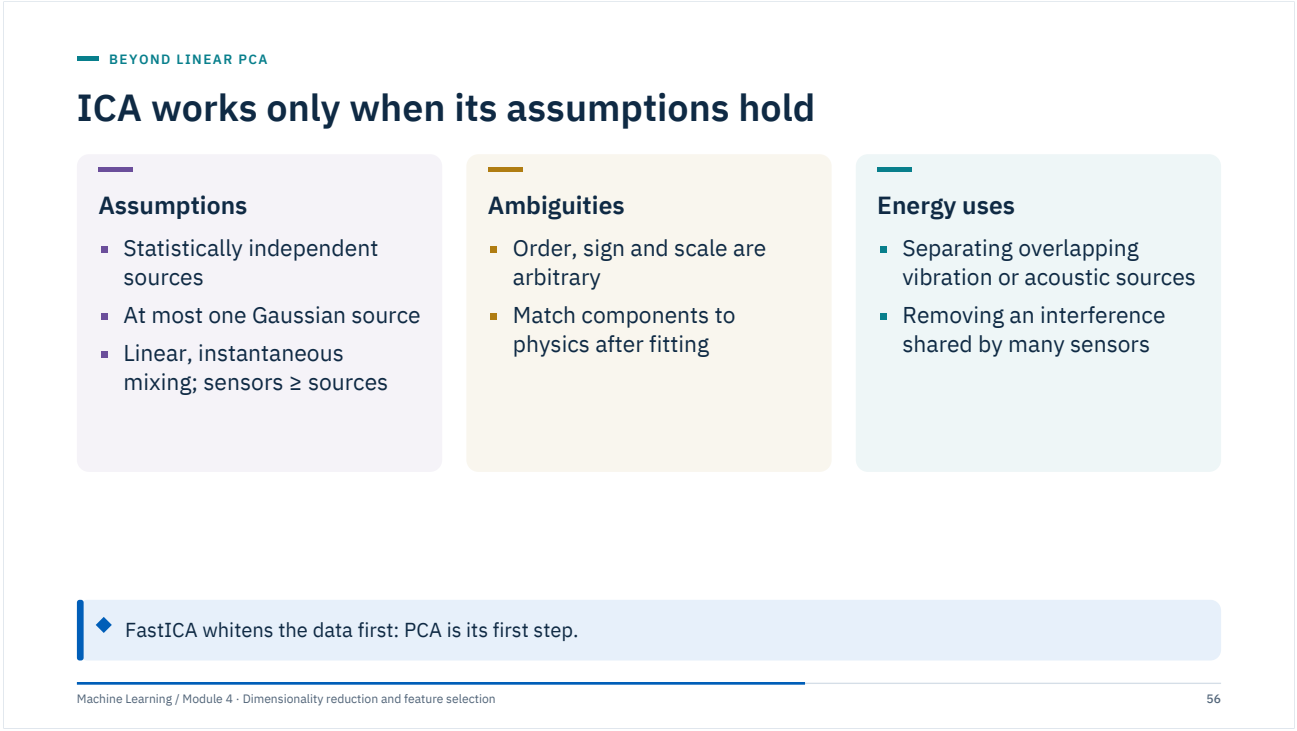
Machine Learning / Module 4 · Dimensionality reduction and feature selection 55

Three synthetic sources: a sinusoid (rotating equipment), a square wave (a cycling valve) and a sawtooth (pressure build-up and bleed-off) [3 min]. Three sensors each record a different linear mixture (second column). PCA (third column) finds uncorrelated directions ordered by variance, but its components are still mixtures: its best match to any source has a correlation of 0.77 to 0.87. FastICA (fourth column) recovers all three sources with absolute correlation 1.00.

Why: decorrelation removes second-order dependence only; ICA looks for directions whose projections are as non-Gaussian, and therefore as independent, as possible. Mixtures of independent signals are closer to Gaussian than the signals themselves (the central limit theorem), so maximizing non-Gaussianity unmixes them.

This is the classic cocktail-party problem. Real separation is harder: noise, delays and echoes break the instantaneous linear mixing assumption. Transition: the assumptions in full.

ICA works only when its assumptions hold



ICA is powerful when its assumptions are plausible and misleading when they are not [3 min]. Independence is stronger than zero correlation. If two sources are both Gaussian, any rotation of them is equally independent, so they cannot be separated. The mixing must be linear and instantaneous; propagation delays between sensors break it.

The output is ambiguous up to order, sign and scale: FastICA returns components in no particular order and with unit variance. Matching a component to a physical source is an interpretation step, using frequency content, timing or known events.

Possible energy uses, stated as ideas to test rather than proven results: separating pump and motor signatures in multi-accelerometer vibration records, or isolating an interference common to many channels.

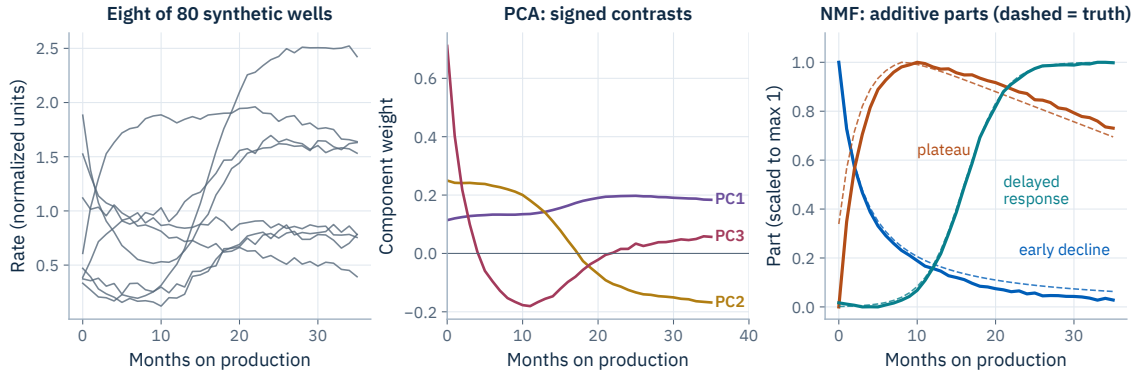
In scikit-learn: FastICA(n_components=k, whiten="unit-variance", random_state=0). Fix the seed, since the solution depends on initialization, and compare runs. Transition: another constraint, non-negativity.

NMF builds profiles from additive parts

BEYOND LINEAR PCA

NMF builds profiles from additive parts

Non-negative data (rates, spectra, concentrations) give non-negative parts and weights.



Synthetic production profiles (seed 9); NMF with k = 3 (seed 0); PCA with k = 3 shown for contrast.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 57

Eighty synthetic wells, each a non-negative mixture of three shapes: an early decline, a plateau followed by slow decline, and a delayed response such as pressure support [3 min].

PCA components (middle) are signed contrasts: PC3 is positive early and negative around month 10. Mathematically fine, but no well produces "negative plateau". NMF (right) factorizes X ≈ W H with W and H constrained to be non-negative, so each well is an additive combination of non-negative parts. Its three parts closely follow the true shapes (dashed).

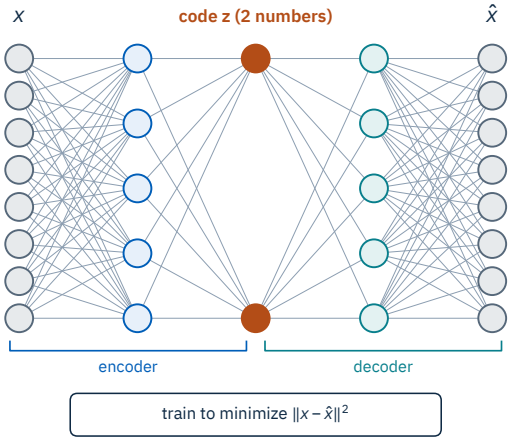
Parts-based representations are easier to explain: "this well is 70% early decline and 30% delayed response". Limits: the solution depends on initialization and k, is not unique, and has no natural ordering of components. In scikit-learn: NMF(n_components=k, init="nndsvda", random_state=0). Choose k by reconstruction error on held-out wells and by whether the parts make physical sense. Transition: a nonlinear bottleneck.

Autoencoders learn a nonlinear bottleneck

BEYOND LINEAR PCA

Autoencoders learn a nonlinear bottleneck

- Encoder: $x \to$ short code z; decoder: $z \to \hat{x}$
- Linear layers with squared error span the same subspace as PCA
- Nonlinear layers can follow curved structure, at a cost in data and care
- Reconstruction error again serves as an anomaly score



Schematic. Eight inputs, a two-number code and eight outputs.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 58

A preview of the neural-network course [3 min]. An autoencoder is a network trained to reproduce its input through a narrow middle layer, the code z. The encoder maps x to z; the decoder maps z back to $\hat{x}$; training minimizes the squared reconstruction error.

With linear layers and squared error, the best code spans the same subspace as the first k principal components (a classical result by Baldi and Hornik, 1989), although the code axes need not be orthogonal. So PCA is the linear autoencoder. Nonlinear activations let the code follow curved manifolds, like kernel PCA but with an explicit encoder for new data.

The price: many parameters, training choices, and more data to avoid memorizing the training set. Start with PCA as the baseline, and require the autoencoder to beat it on held-out reconstruction or task error.

Cross-references: NN Day 2 introduces autoencoders, NN Day 4 covers convolutional autoencoders for images, and Module 5 uses reconstruction error for anomaly detection.

Each goal points to a different method

PRACTICE

5 min Pairs

Each goal points to a different method

Name one method for each goal, and one check you would run:

1. Summarize 20 correlated rig channels for storage within an error budget
2. A 2-D view that best separates three labelled facies for a report
3. Unmix pump and motor vibration recorded by three accelerometers
4. Describe non-negative production profiles as a few typical shapes
5. One unsupervised score that separates two concentric groups in a cross-plot

Give 3 minutes, then take one answer per item [5 min].

1. PCA on standardized channels, k chosen by held-out reconstruction error. Check: per-channel error in engineering units against the budget, on a later period.

2. LDA, which gives at most C – 1 = 2 components for three facies. Check: fit on training wells only and show held-out wells projected into the map; colouring the training wells alone is circular.

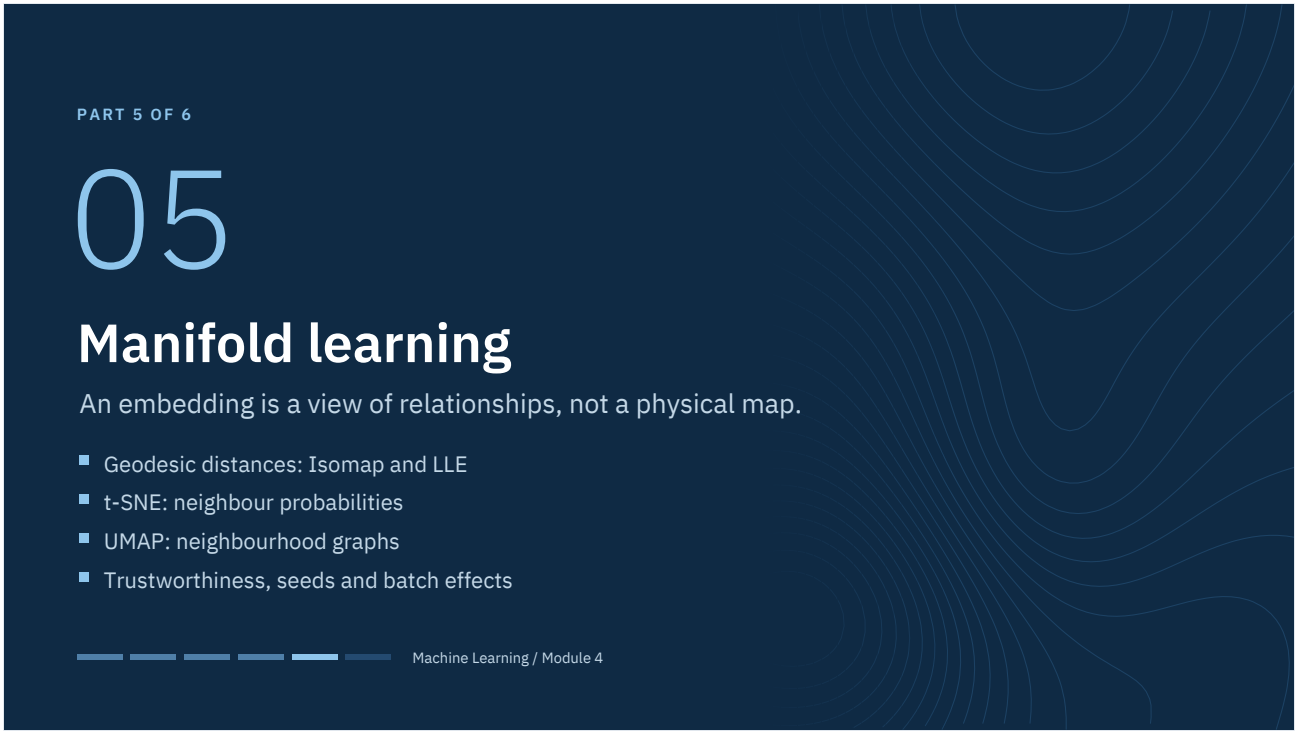
3. ICA (FastICA), with at least as many sensors as sources. Check: component spectra against known pump and motor frequencies; stability across seeds.

4. NMF, because rates are non-negative and parts should add. Check: held-out reconstruction and whether parts resemble known decline behaviours.

5. Kernel PCA with an RBF kernel (or Isomap). Check: sensitivity to γ, and whether the score separates the groups on new data.

Common errors: choosing t-SNE for item 1 (no reconstruction), or PCA for item 2 (ignores the labels).

Manifold learning



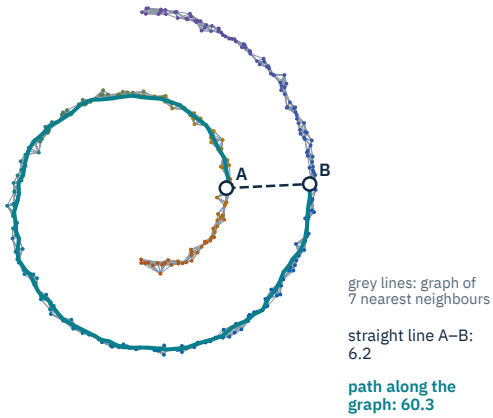
Part 5 runs about 50 minutes and prepares Lab 2. Use this transition to prevent interpreting between-island distances as measured dissimilarity. Ask learners what a projection can distort even if local neighbourhoods look plausible: distances between groups, sizes and densities of groups, and orientation. The part ends with an exercise on a map that hides a batch effect. [1 min]

Curved data need distances along the surface

MANIFOLD LEARNING

Curved data need distances along the surface

- A and B are 6.2 apart in a straight line, 60.3 along the data
- A k-nearest-neighbour graph approximates the surface
- Shortest paths on the graph estimate geodesic distance
- Isomap then embeds these distances with classical MDS



Synthetic noisy spiral (seed 0, 400 points); 7-nearest-neighbour graph; scipy shortest path. Colour = position along the spiral.

The spiral is a two-dimensional cross-section of the Swiss roll on the next slide [3 min]. Points A and B lie on adjacent turns: in a straight line they are 6.2 apart, closer than many points on the same turn. Along the data, the only route is around the spiral: 60.3.

A manifold learner wants distances along the data, called geodesic distances. It cannot see the spiral, but it can connect each point to its nearest neighbours; short hops are trustworthy because locally the data look flat. The shortest path through that graph (teal) approximates the geodesic distance.

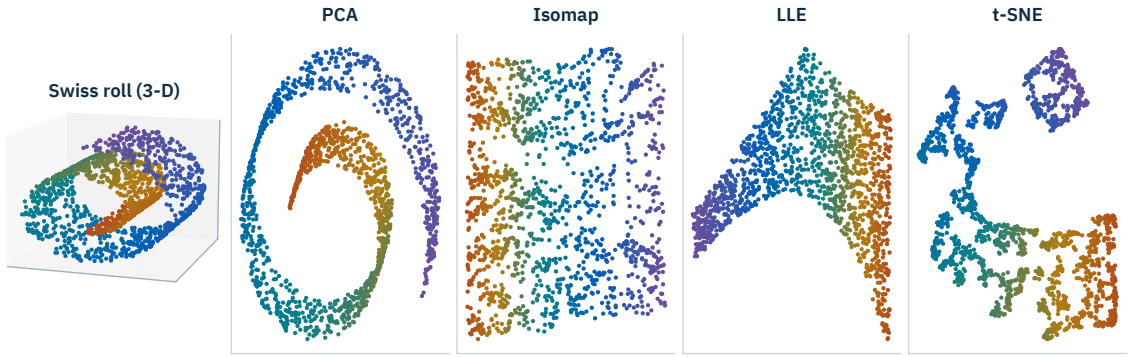
Isomap computes all such graph distances and then finds a low-dimensional layout whose Euclidean distances match them (classical multidimensional scaling). If the number of neighbours is too large, edges jump between turns and short-circuit the structure. Ask: what would happen with more neighbours? (Computed on this spiral: the A–B graph distance is 60.3 with 7 neighbours, 31.1 with 30 as shortcuts appear, and 6.2, the straight line, with 60; the unrolled map would fold.)

Methods unroll the Swiss roll differently

MANIFOLD LEARNING

Methods unroll the Swiss roll differently

Colour shows position along the roll; it is never used in fitting.



scikit-learn make_swiss_roll (1,500 points, noise 0.05, seed 0); Isomap k = 10, LLE k = 12, t-SNE perplexity 30.

Compare the four maps with the colour gradient [4 min].

PCA finds the two directions of largest spread, which are the spiral's cross-section, so it shows the roll edge-on: points far apart along the roll can land close together.

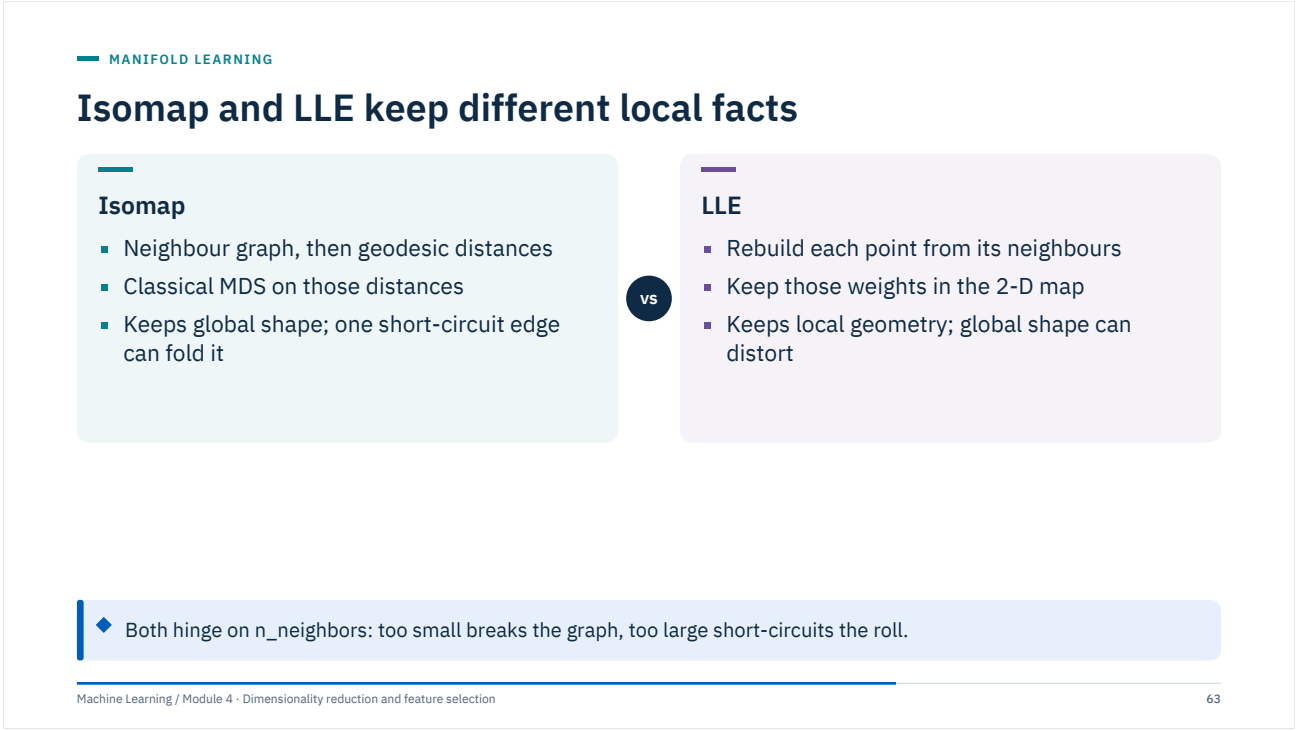
Isomap unrolls the sheet into a rough rectangle with the colour gradient running cleanly across it, because it preserves distances along the neighbourhood graph: its first axis follows position along the roll almost exactly (correlation 0.99), and the spacing matches the true distance along the surface to within about 5%. The streaky, patchy look at the purple end is thinner sampling, not mixed-up neighbours: make_swiss_roll spreads points evenly by angle, so the outer end of the roll gets about half as many points per unit length as the inner end.

LLE also preserves the order of colours but distorts the global shape into a wedge; it keeps local geometry, not distances.

t-SNE keeps colour neighbourhoods intact but tears the sheet into several pieces and arranges them freely: local structure is kept, global shape is not.

No method is "best" in general; each preserves what its objective says. On real data there is no colour gradient to check against, which is why trustworthiness and stability checks (later in this part) matter. Transition: Isomap and LLE side by side.

Isomap and LLE keep different local facts



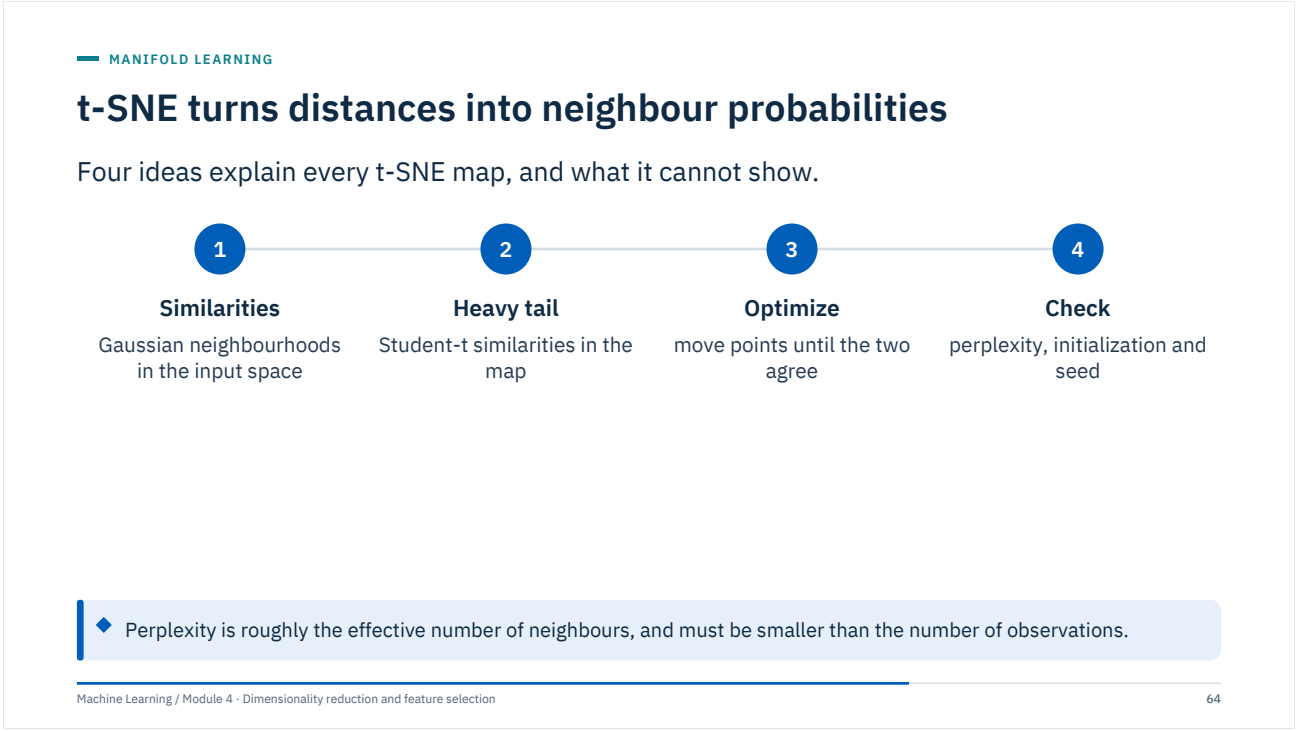
Isomap (Tenenbaum, de Silva and Langford, 2000) keeps approximate geodesic distances between all pairs, so it can recover the global layout of a single, well-sampled sheet [3 min]. Its weakness is topological: a single edge across a gap changes many shortest paths at once.

LLE (Roweis and Saul, 2000) describes each point as a weighted combination of its neighbours and finds a low-dimensional layout where the same weights still work. It only constrains local geometry, so the global shape can bend or collapse; modified and Hessian variants (method="modified", "hessian") are more stable.

Both are available in sklearn.manifold (Isomap, LocallyLinearEmbedding) and both provide approximate transform() for new points. Both scale poorly beyond tens of thousands of points without approximate neighbours.

Practical rule: sweep n_neighbors (for example 5, 10, 20, 40), check the graph is connected, and compare maps with trustworthiness. Transition: t-SNE, which keeps only neighbourhoods.

t-SNE turns distances into neighbour probabilities



The four steps [3 min]. First, t-SNE converts distances in the original space into local similarities: each point spreads a Gaussian over its neighbours, with a width chosen per point so that the effective number of neighbours equals the perplexity. Second, it represents similarities in the low-dimensional map with a heavy-tailed Student-t distribution. Third, it optimizes the map positions so that important local relationships are retained. Fourth, inspect sensitivity to perplexity, initialization and random seed.

The heavy-tailed Student-t distribution helps address crowding in the low-dimensional map: moderate distances in high dimensions can be represented as larger distances in 2-D, which separates groups visually. The optimization is nonconvex, so different starts can end in different layouts.

Perplexity is related to the effective neighbourhood size and must be less than the number of observations. Transition: the objective function that makes local neighbours matter most.

The t-SNE objective is asymmetric

MANIFOLD LEARNING

The t-SNE objective is asymmetric

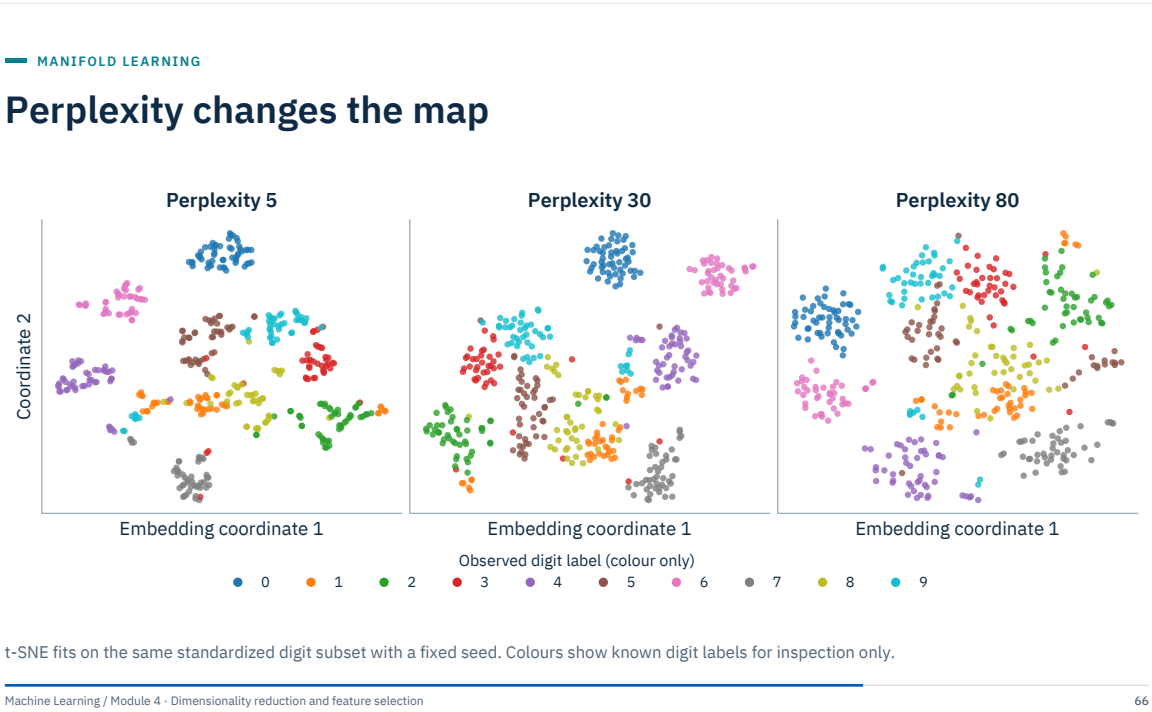
$$\mathrm{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- P describes high-dimensional similarities
- Q describes similarities in the embedding
- Missing an important original neighbour incurs a strong penalty

The Kullback–Leibler divergence compares the two sets of similarities [2 min]. p_ij is large for true neighbours; q_ij is the similarity of the same pair in the map. When p_ij is large and q_ij small (true neighbours placed far apart), the term p_ij log(p_ij / q_ij) is large: t-SNE works hard to keep neighbours together. When p_ij is small and q_ij large (non-neighbours placed close), the penalty is small: t-SNE cares much less about keeping far points far.

This optional equation supports the interpretation that local neighbourhoods matter strongly. Low objective values across different preprocessing or perplexity settings are not automatically comparable evidence for scientific validity. Avoid deriving all bandwidth optimization details unless the audience requests them.

Perplexity changes the map



Known labels are not inputs to these unsupervised fits [3 min]. Compare nearby neighbourhoods and separation, not absolute axis values.

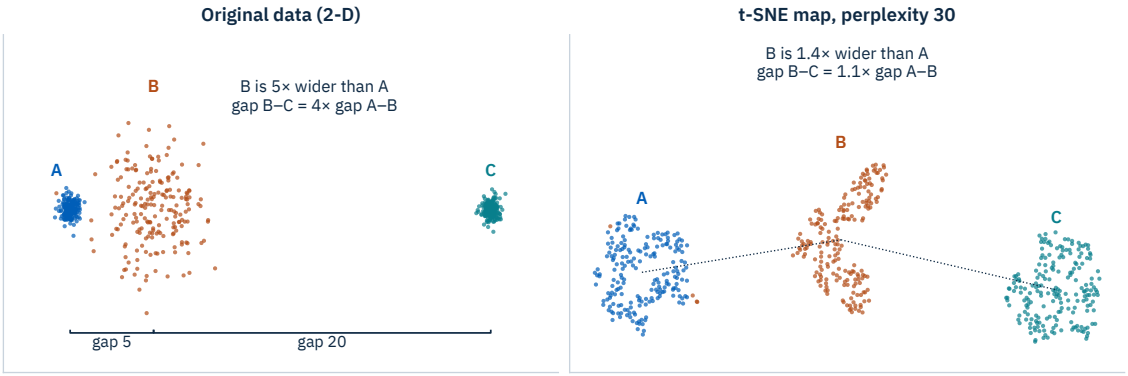
At perplexity 5, each point considers only a handful of neighbours: many small fragments and strings, with some digits split into several pieces. At 30, most digits form compact islands, although 1 and 8 still mix. At 80, the islands are broader and less separated.

Larger perplexity considers broader neighbourhoods but does not guarantee a truer global map. There is no correct perplexity; a finding that appears only at one setting is fragile. Report the settings, and show at least two perplexities when a map supports a decision. Transition: what sizes and gaps in the map mean.

Island sizes and gaps in t-SNE are not measurements

MANIFOLD LEARNING

Island sizes and gaps in t-SNE are not measurements



Synthetic 2-D clusters (seed 0): spreads 0.3, 1.5 and 0.3 per axis, centres at 0, 5 and 25; t-SNE seed 0. Spread = RMS distance to the cluster centre.

The truth is known here because the data are two-dimensional [3 min]. Cluster B is five times wider than A (RMS spread 2.1 versus 0.4), and the gap between B and C is four times the gap between A and B (20 versus 5).

In the t-SNE map, B is only 1.4 times wider than A, and the B–C gap is only 1.1 times the A–B gap. t-SNE adapts its bandwidth to local density, so dense clusters are expanded and sparse ones contracted; and the heavy tails compress large distances. Both distort sizes and gaps.

Wattenberg, Viégas and Johnson (Distill, 2016) show many more such cases, including maps of pure noise that show apparent clusters.

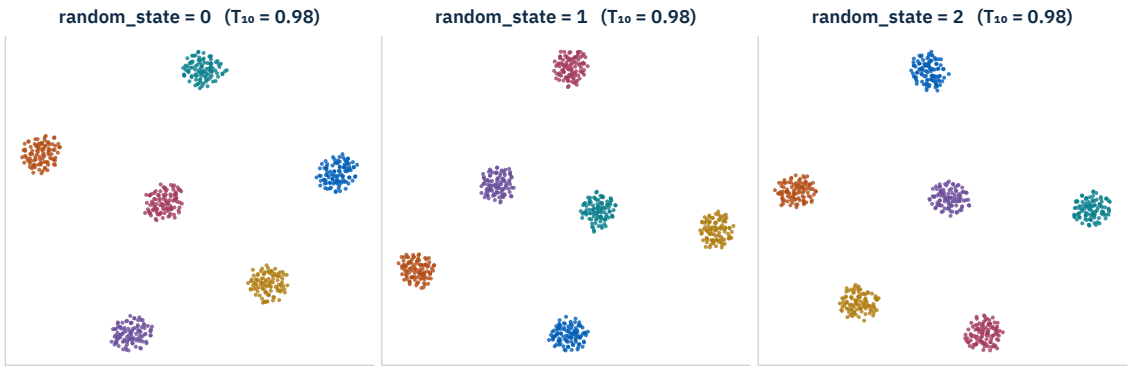
Rule: read a t-SNE map for which points are neighbours; never measure island sizes, densities or distances between islands on it. If size or separation matters, compute it in the original feature space. Transition: the layout itself can change from run to run.

Random initialization changes the layout, not the neighbours

MANIFOLD LEARNING

Random initialization changes the layout, not the neighbours

Same data and perplexity; only random_state differs. Trustworthiness T_{10} is 0.98 in all three.



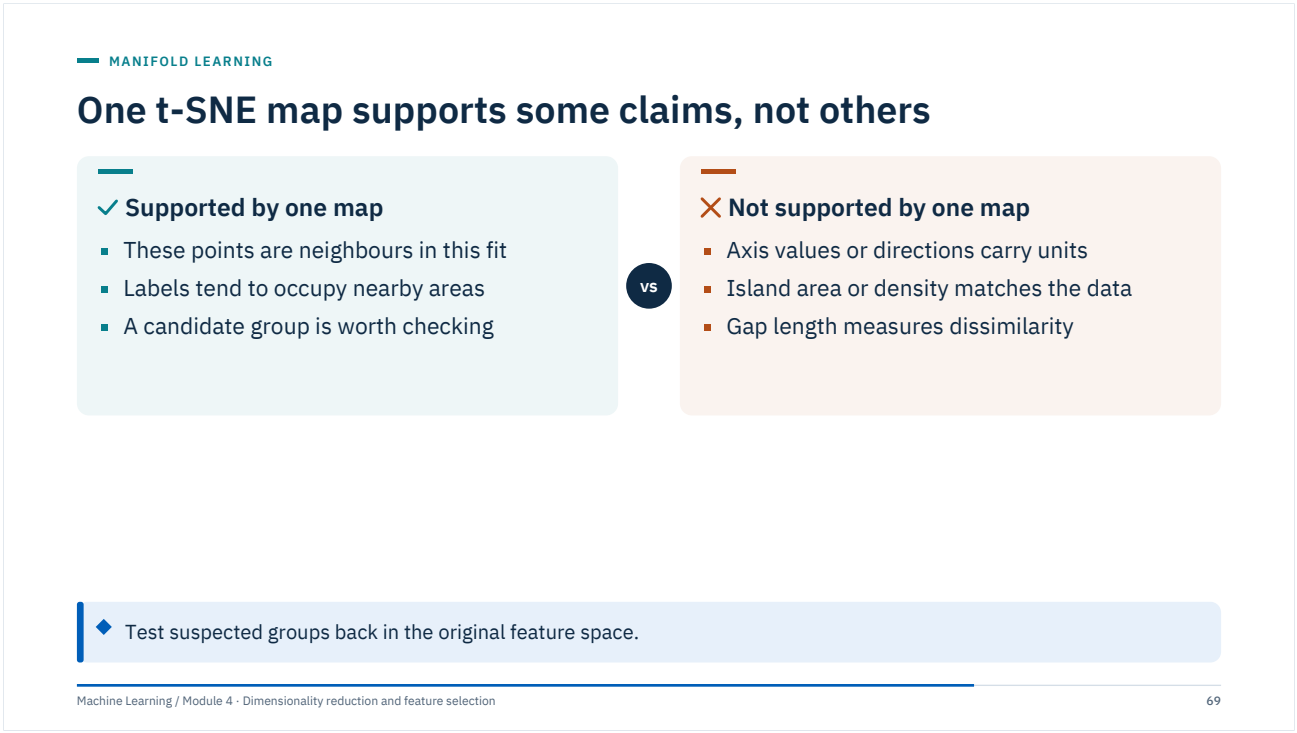
Six synthetic 10-D Gaussian clusters (make_blobs seed 11); t-SNE perplexity 30, init="random". Colours show generating clusters, not used in fitting.

Three runs with init="random" and seeds 0, 1 and 2 [3 min]. The six islands are the same and equally well separated, and trustworthiness at 10 neighbours is 0.98 in every run, yet their arrangement differs: which islands sit next to each other and where they lie is decided by the random start, not by the data.

Since scikit-learn 1.2 the default is init="pca", which starts from the PCA layout. On these data, PCA initialization gives nearly identical maps for all three seeds. That makes a run repeatable, which is useful for reports, but repeatability is not truth: it only fixes one of many equally good layouts.

A more informative stability check changes the data: rerun on bootstrap samples or on different wells, and at two perplexities. Claims that survive those changes deserve more trust. Results can also differ slightly across library versions and hardware. Transition: supported and unsupported claims.

One t-SNE map supports some claims, not others



Axes have no direct physical units [3 min]. Island area and apparent density need not match the original data. Long gaps between islands are not reliable quantitative distances. Test suspected groups back in the original feature space.

Ask learners to write one supported observation and one unsupported claim about the perplexity figure a few slides back. Supported: particular labels tend to occupy neighbouring areas in this fit. Unsupported: one class is exactly twice as different from another because of a visible gap.

A useful habit: every statement about a map in a report should name the check that supports it, such as "confirmed by comparing log distributions of the two groups" or "stable across seeds and two perplexities". Transition: the current API.

Current t-SNE code uses max_iter, not n_iter

MANIFOLD LEARNING

Current t-SNE code uses max_iter, not n_iter

```python
from sklearn.manifold import TSNE

embedding = TSNE(
    n_components=2, perplexity=30,
    init="pca", learning_rate="auto",
    max_iter=1000, random_state=42
).fit_transform(Z)
```

Z is an explicitly prepared feature matrix with more than 30 rows.

The iteration limit is set with max_iter [2 min]. Older code examples use n_iter, which recent scikit-learn releases removed, so such code raises an error. init="pca" and learning_rate="auto" are the defaults in current versions, written out here for clarity.

Z should be standardized (or otherwise deliberately scaled) numeric features. Optionally reduce noisy high-dimensional data with PCA first, for example to 30 to 50 components, then inspect whether neighbourhoods remain meaningful.

Do not train a predictive model on this embedding and claim a standard new-sample pipeline without an out-of-sample method: TSNE has no transform() for new rows. Use t-SNE for exploration, and PCA, selection or UMAP when new data must be mapped. Transition: UMAP.

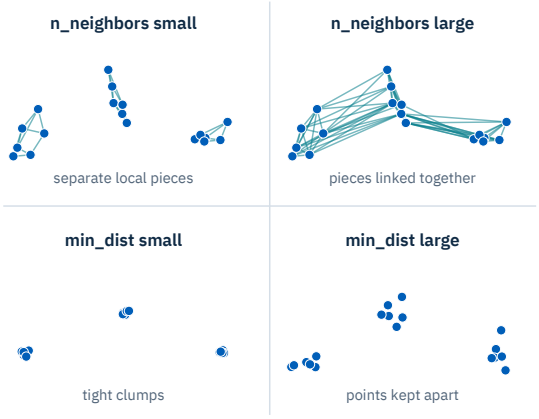
Reference: https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html

UMAP starts from a neighbourhood graph

MANIFOLD LEARNING

UMAP starts from a neighbourhood graph

- Build weighted local relationships in the input space
- Optimize a layout that approximates those relationships
- n_neighbors sets the neighbourhood scale of the graph
- min_dist sets how tightly points may pack in the map



Schematic, not UMAP output; umap-learn is an optional package (requirements-labs-optional.txt).

Machine Learning / Module 4 · Dimensionality reduction and feature selection 71

UMAP derives its construction from manifold and fuzzy-set ideas, but a conceptual graph explanation is sufficient for this course (McInnes, Healy and Melville, 2018) [3 min].

Top row: n_neighbors controls the neighbourhood scale used to construct the graph. Small values connect each point only to its closest neighbours, emphasizing fine local pieces; large values link pieces together and let broader structure influence the layout.

Bottom row: min_dist influences how tightly points can pack in the embedding. Small values give tight clumps, which can look like strong clusters; larger values keep points apart and show more of the spread.

Neither parameter directly specifies a desired number of clusters. The choice of metric can be as important as the embedding settings. Treat n_neighbors like t-SNE's perplexity: sweep it and report it.

Graph cross-entropy balances attraction and separation

MANIFOLD LEARNING

Graph cross-entropy balances attraction and separation

$$-\sum_{i<j}[v_{ij}\log w_{ij} + (1-v_{ij})\log(1-w_{ij})]$$

- v represents input-graph membership strength
- w represents membership implied by embedded distance
- Practical optimization uses sampling rather than all pairs on every step

Machine Learning / Module 4 · Dimensionality reduction and feature selection 72

The first term attracts: where the input graph has a strong edge (v_ij near 1), the map is penalized unless the points are close (w_ij near 1). The second term repels: where the input graph has no edge (v_ij near 0), the map is penalized if the points are close [2 min]. Unlike t-SNE's KL divergence, this explicit repulsive term penalizes placing non-neighbours close together. Claims that UMAP preserves more global structure than t-SNE are debated: Kobak and Linderman (2021) showed that much of the difference comes from initialization, so use an informative initialization for either method.

This is a schematic cross-entropy objective, omitting terms constant in embedding coordinates. The implementation uses negative sampling and approximations. Do not present the displayed expression as a literal description of every optimization step.

UMAP supports an explicit future-data transform

MANIFOLD LEARNING

UMAP supports an explicit future-data transform

```
import umap

reducer = umap.UMAP(
    n_neighbors=15, min_dist=0.1,
    metric="euclidean", random_state=42
)
Z_train = reducer.fit_transform(X_train_scaled)
Z_new = reducer.transform(X_new_scaled)
```

Evaluate whether the fitted representation transfers to new wells.

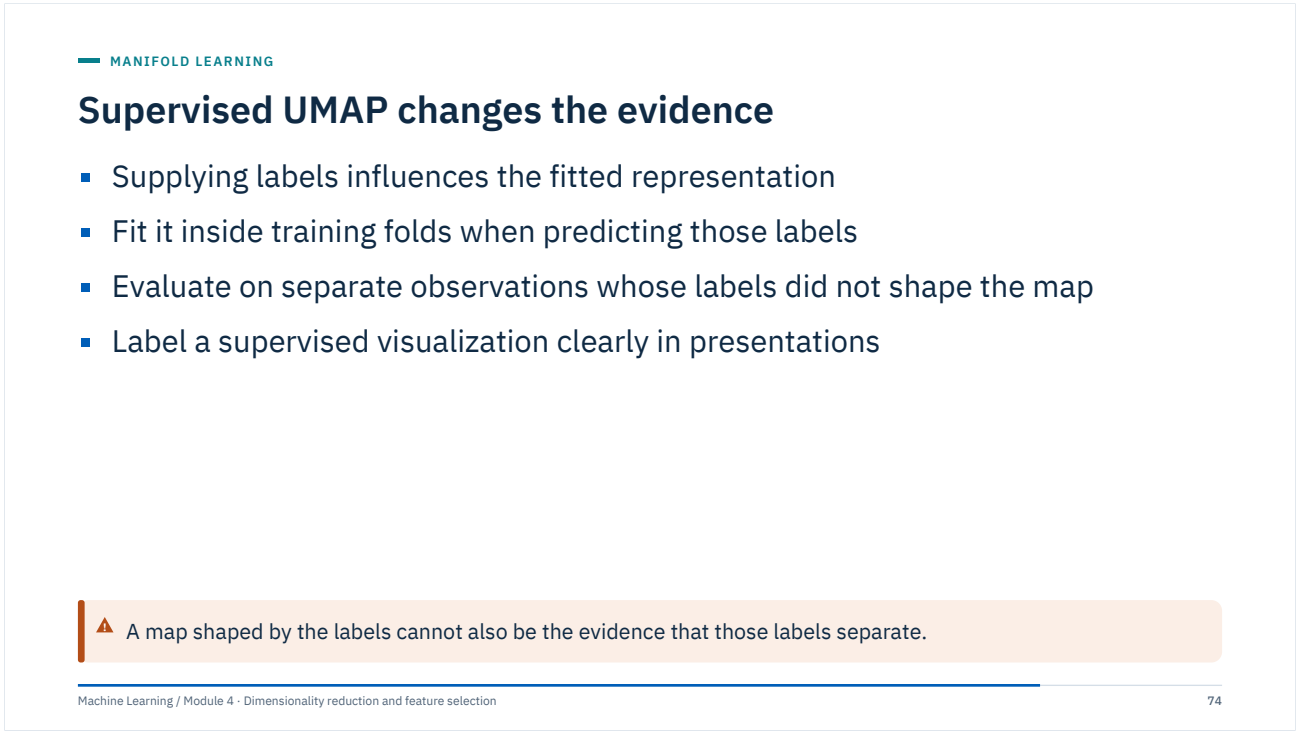
The optional package is umap-learn, imported as umap. It is listed in requirements-labs-optional.txt, not in the core lab requirements; install it only if your organization's environment allows it (pip install umap-learn) [2 min].

New data still needs the same fitted imputer and scaler: X_new_scaled must be produced by the scaler fitted on the training wells. UMAP can transform new observations but cannot guarantee meaningful placement far outside the training population; a new well from a different field may land between islands for reasons unrelated to geology.

Setting random_state makes runs reproducible but disables some parallelism in umap-learn, so fitting can be slower. As with t-SNE, report n_neighbors, min_dist and the metric with every figure. Transition: what changes when labels are supplied.

Reference: https://umap-learn.readthedocs.io/en/latest/parameters.html

Supervised UMAP changes the evidence



umap-learn accepts a target y in fit, and the labels then pull same-class points together [2 min]. That can produce beautiful, well-separated maps.

Clusters coloured by the same labels that shaped the embedding are not independent evidence that those classes are naturally separable. The map has been told the answer. The honest test: fit on training wells, transform held-out wells, and see whether their classes separate.

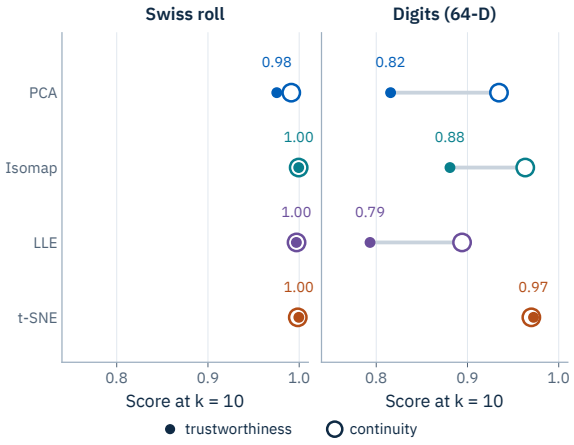
Semi-supervised modes, where only some rows carry labels, also require care with which labels enter each fold. The same logic applies to LDA from Part 4 and to any supervised feature selection from Part 2: labels used to build a representation cannot also validate it. Ask: have you seen a supervised map presented as an unsupervised discovery?

Trustworthiness scores how faithful neighbourhoods are

MANIFOLD LEARNING

Trustworthiness scores how faithful neighbourhoods are

- Trustworthiness: are map neighbours true neighbours?
- Continuity: are true neighbours kept in the map?
- Swiss roll: PCA 0.98, the other methods 0.997 or more
- Digits: t-SNE 0.97, Isomap 0.88, PCA 0.82, LLE 0.79



sklearn.manifold.trustworthiness at k = 10; continuity is the same function with the arguments swapped. Swiss roll seed 0; 600 standardized digit images (seed 42).

Trustworthiness T(k) penalizes "intruders": points that are among the k nearest neighbours in the map but were not neighbours in the original space, weighted by how far away they really were. It is 1 when no intruders exist [4 min]. Continuity penalizes the opposite error: true neighbours pushed away. scikit-learn provides trustworthiness(X, X_embedded, n_neighbors=k); calling it with the arguments swapped gives continuity.

Results at k = 10. Swiss roll: PCA 0.98, the others 0.997 to 1.000; even the edge-on PCA view keeps most small neighbourhoods. Digits (64 features): t-SNE 0.97, Isomap 0.88, PCA 0.82, LLE 0.79, with continuity higher than trustworthiness for the linear and graph methods.

Use it to compare methods and settings on the same rows and the same k; report k. High trustworthiness does not validate global distances or cluster sizes, and it does not measure usefulness for a task. In Lab 2 you will add it to every map.

A comparison needs a fixed protocol

MANIFOLD LEARNING

A comparison needs a fixed protocol

Dimension	PCA	t-SNE	UMAP
Geometry	Linear variance	Local similarities	Neighbourhood graph
Interpretation	Directions and loadings	Local visual neighbourhoods	Local visual neighbourhoods
Reproducibility	Usually stable up to sign	Check seeds and settings	Check seeds and settings
Validation	Reconstruction or task score	Neighbourhoods and stability	Neighbourhoods, stability, task score

◆ Same rows, same scaling, recorded settings and at least two seeds for every method.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 76

This protocol is what Lab 2's comparison route asks for [3 min]. Same rows, same scaling, same features for every method; settings recorded; at least two seeds and two neighbourhood settings for the nonlinear methods; trustworthiness at a stated k; and, if the representation feeds a model, the held-out task score with and without it.

Avoid blanket runtime rankings. Record sample count, dimension, hardware, implementation and settings before claiming one method is faster. Exact outputs can vary across package versions and numeric environments.

The comparison notebook also computes silhouette scores of known facies in each 2-D map. Treat those with care: they measure how separated the labels look in the map, which t-SNE and UMAP can exaggerate, not how separable the facies are in the logs. Transition: an exercise on reading maps with metadata.

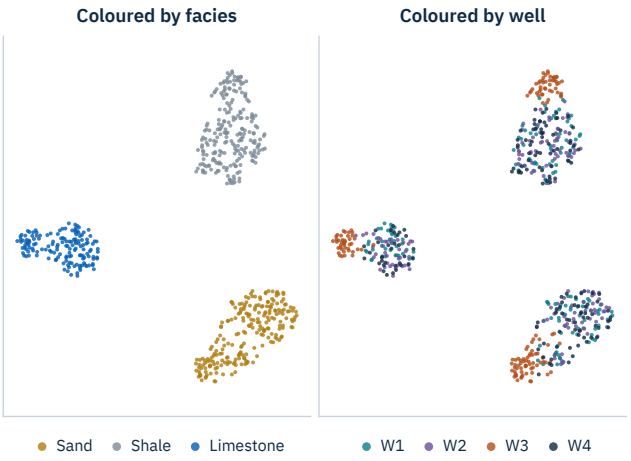
Colour the map by metadata before trusting it

PRACTICE

5 min · Pairs

Colour the map by metadata before trusting it

1. What does the left panel suggest?
2. What does the right panel add?
3. Which check in the original logs comes next?
4. What would you change before clustering?



Synthetic logs from four wells (seed 21); t-SNE on standardized GR, RHOB, NPHI and PEF.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 77

The same t-SNE map coloured two ways [5 min]. Give 3 minutes in pairs.

1. Left: three well-separated islands that match the facies labels; the logs distinguish sand, shale and limestone.

2. Right: inside every facies island, the points from well W3 form their own sub-island at the edge. Something makes W3's logs systematically different in every facies. That is a batch effect: here a density–neutron offset injected into W3, as a calibration or environmental correction problem might produce.

3. Compare per-well distributions of each log within a facies (box plots or cross-plots of RHOB against NPHI by well). A shift in RHOB and NPHI only in W3 points to the tool or its corrections, not to geology.

4. Fix or correct the offset if it is an acquisition artefact (with petrophysical justification), or normalize per well with care; then cluster. Never label the W3 sub-islands as a new facies without evidence.

Transition to Lab 2, where colouring by metadata after fitting is a required step.

Lab 2: nonlinear maps and a fair comparison

LABORATORY

60 min

Lab 2: nonlinear maps and a fair comparison

- ☐ Route A (02_tsne): production data; perplexity 5 to 50; two seeds with init="random"
- ☐ Route B (04_comparison): PCA and t-SNE on petro.csv under one protocol
- ☐ Add `trustworthiness(X, emb, n_neighbors=10)` to every map
- ☐ Colour by metadata after fitting; write one supported and one unsupported claim
- ☐ UMAP (03_umap) is optional: umap-learn is in requirements-labs-optional.txt

NOTEBOOK

`exercises/02_tsne/tSNE_Production_Student.ipynb`

EVIDENCE TO SUBMIT

- Maps at two perplexities and two seeds, each with its trustworthiness
- One supported conclusion, one unsupported interpretation, one next check

Machine Learning / Module 4 · Dimensionality reduction and feature selection 78

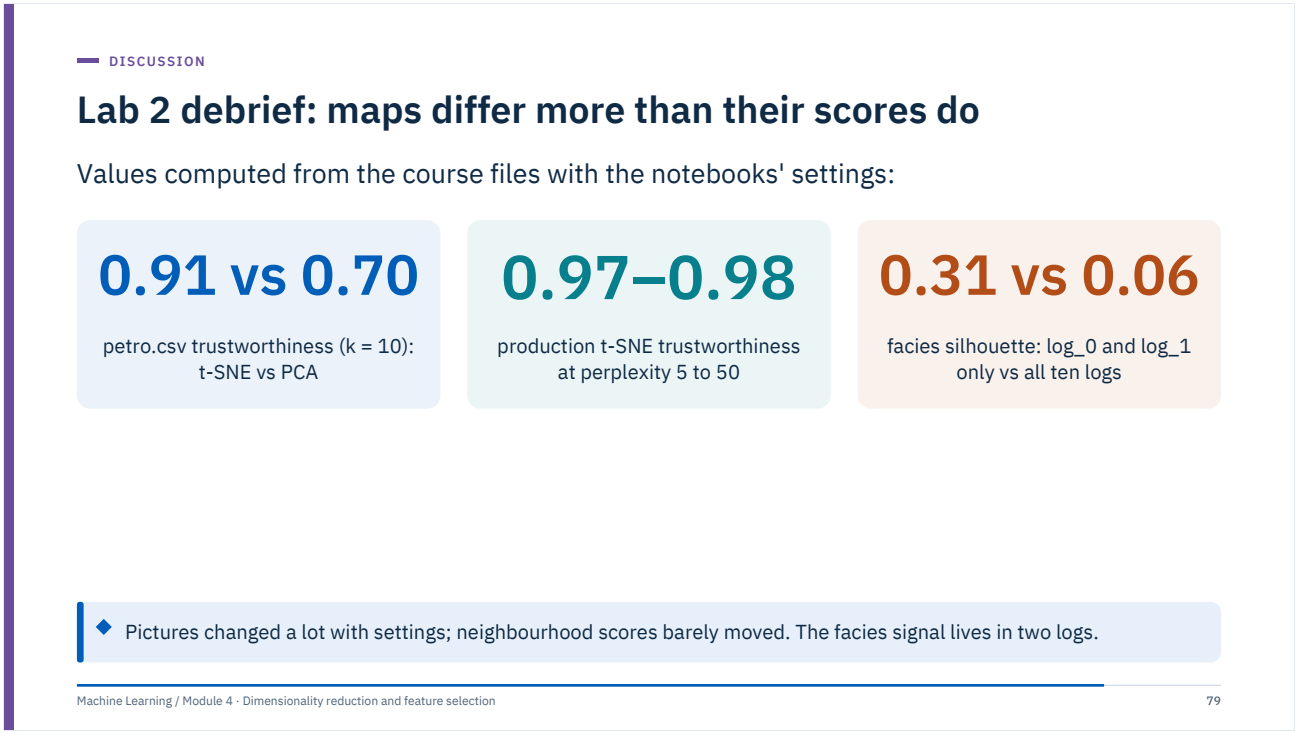
Timing: 60 minutes (15:30–16:30), then the debrief. Let learners choose a route: Route A (exercises/02_tsne) for t-SNE practice on 725 production wells with 10 features; Route B (exercises/04_comparison/DR_Comparison_Student.ipynb) for a protocol-driven comparison on 1,000 synthetic petrophysical samples with 10 logs and 3 facies. Solutions are the matching *_Solution.ipynb files. The UMAP notebook (exercises/03_umap) and the UMAP cells in Route B are skipped automatically when umap-learn is missing; they are optional extensions. Route A draws its correlation heatmap with seaborn, which is in requirements-labs.txt; Route B and the UMAP notebook do not use it.

Both solutions compute trustworthiness(X, emb, n_neighbors=10) for every map and repeat t-SNE with two seeds (init="random") at two perplexities; Route B also repeats UMAP with two seeds and two n_neighbors values. In the student notebooks these are TODO cells; the import is already in the first cell. Facies labels never enter the inputs; every map is labelled exploratory.

Circulate with prompts: "What changed between your two seeds, and what did not?" "Which claim would you defend to a petrophysicist?"

Common errors: forgetting to standardize (depths and footages in feet dominate Route A); comparing KL divergence across perplexities; reading silhouette of facies in a t-SNE map as proof of separability; tuning perplexity until the picture looks good.

Lab 2 debrief: maps differ more than their scores do



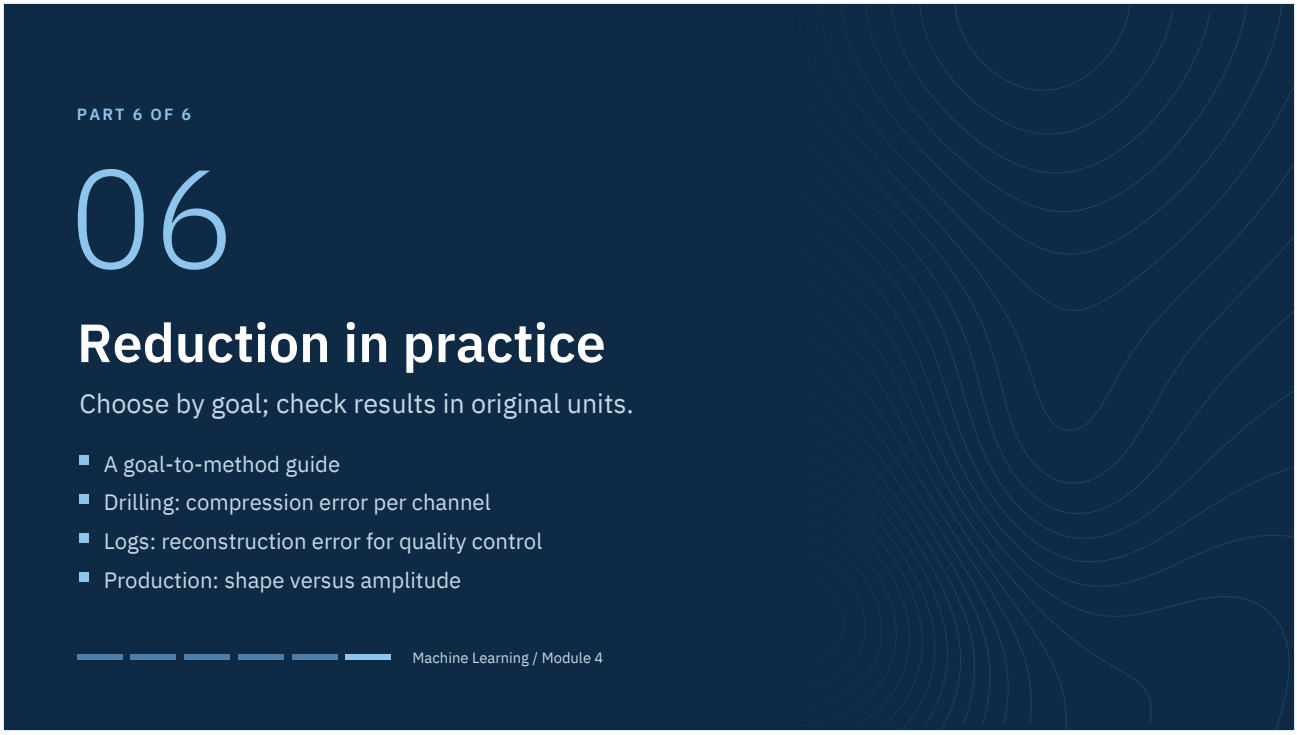
Numbers from the solution notebooks run on the course files (random_state 42, scikit-learn 1.9); the two-log silhouette was computed separately from petro.csv [7 min].

Route B (petro.csv): the 2-D PCA keeps 24% of the variance and has trustworthiness 0.70; t-SNE reaches 0.91. The notebook's facies silhouette is 0.06 in the full 10-D space, 0.08 for PCA and 0.18 for t-SNE (0.18 for UMAP when installed). Across two seeds and two neighbourhood settings, t-SNE trustworthiness stays between 0.90 and 0.92 and UMAP's between 0.83 and 0.85. Do not read the higher map values as "facies are separable": maps exaggerate separation. The data explain why: facies differ only in log_0 and log_1 (means near ±1); the other eight logs have nearly identical means across facies. Using just those two logs, silhouette is 0.31. Feature selection beats any embedding here, which links back to Parts 1 and 2.

Route A (production): t-SNE trustworthiness stays between 0.97 and 0.98 for perplexities 5 to 50 and for two random seeds, while the pictures differ visibly; PCA gives 0.82.

Ask each group for one supported conclusion and one claim they withdrew after checking. Transition to Part 6.

Reduction in practice



Part 6 is a short synthesis (about 15 minutes) after the Lab 2 debrief. It turns the day into a decision guide and walks through three energy applications: compressing drilling channels, screening logs for bad-hole intervals, and mapping production profiles. Each comes with the check in original units that makes it trustworthy. If time is short, keep the decision table and the log QC example, and go straight to the exit check. [1 min]

Choose the method from the goal

REDUCTION IN PRACTICE

Choose the method from the goal

Goal	First choice	Evidence to report
Keep original sensors	Filter, wrapper or embedded selection	Selection frequency across folds
Predict a target	Selection or PCA inside the pipeline	Held-out score vs. no reduction
Compress or QC	PCA, k by held-out error	Error per channel in original units
Visualize neighbourhoods	t-SNE or UMAP, with PCA alongside	Trustworthiness, seeds, metadata colours
Unmix or find parts	ICA or NMF	Physical plausibility of components

Machine Learning / Module 4 · Dimensionality reduction and feature selection 81

Read this as a starting point, not a rulebook [3 min]. Each row pairs a goal with a default method and, more importantly, with the evidence that makes the result defensible.

Keep sensors: selection, because extraction still needs every input. Predict: selection or PCA as a tuned pipeline step, always compared with no reduction on the same grouped folds. Compress or monitor: PCA with k chosen by held-out reconstruction error, reported per channel. Visualize: a nonlinear map shown next to PCA, with trustworthiness, two seeds or settings, and colours from metadata added after fitting. Unmix or find parts: ICA for independent sources, NMF for non-negative additive parts, judged by physical plausibility and stability.

Kernel PCA, Isomap, LDA and autoencoders fit particular cases: curved structure, labelled projection, or a nonlinear bottleneck with enough data. Ask: which row describes your next project?

Compression error must be read channel by channel

REDUCTION IN PRACTICE

Compression error must be read channel by channel

- With k = 3, most channels keep a quarter to a third of their SD as error
- Gas is lost completely at k = 3 (8.5 units, its full SD)
- k = 4 recovers gas: the channel that matters may carry little variance

	k = 1	k = 2	k = 3	k = 4	k = 5
WOB (klbf)	1.8	1.6	1.3	1.3	1.3
Torque (kft·lbf)	1.9	1.9	0.8	0.8	0.8
RPM (rpm)	24	25	4.8	3.1	3.1
ROP (ft/h)	11	11	3.9	3.7	3.4
SPP (psi)	242	95	62	56	51
Flow (gpm)	55	20	16	14	14
ECD (ppg)	0.10	0.04	0.03	0.03	0.02
DiffP (psi)	50	33	25	25	16
Hookload (klbf)	2.4	2.2	2.0	2.0	1.9
Gas (units)	8.2	8.3	8.5	0.08	0.07

Validation RMSE in engineering units (shade = RMSE / channel SD)

Synthetic drilling channels (seed 4); PCA fitted on standardized minutes 0–429; validation RMSE on minutes 430–719 in original units.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 82

Drilling parameter compression, read the way an engineer would [3 min]. Each cell is the validation root-mean-square reconstruction error in the channel's own units; shading is that error divided by the channel's standard deviation.

With three components, the validation RMSE is about 1.3 klbf for WOB, about 60 psi for SPP and 0.03 ppg for ECD: roughly a quarter to a third of each channel's standard deviation. Whether that is acceptable depends on the use: fine for trend storage, not for detecting a 50 psi pressure change.

Gas is the warning: at k = 3 its error (8.5 units) equals its full standard deviation, so the reconstruction is just its mean. Explained variance at k = 3 is about 85% on the training period, which sounds good, but the one channel that often matters most for safety is gone. The fourth component is essentially the gas channel alone.

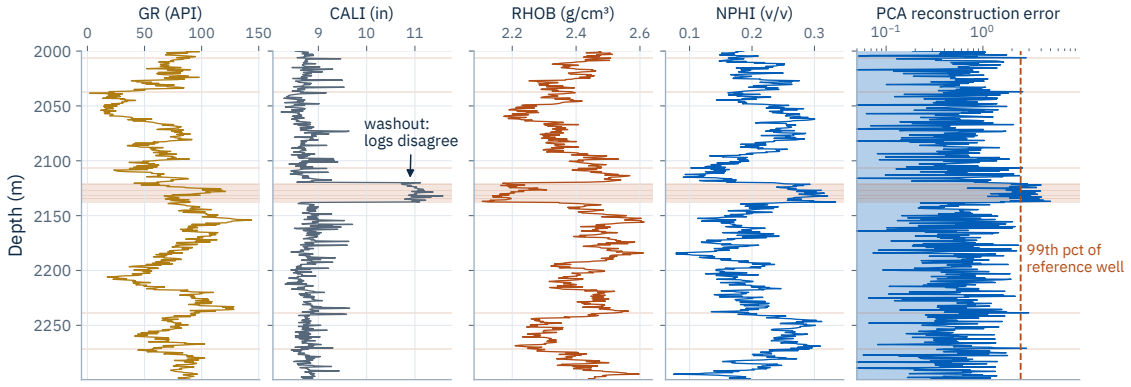
Rule: state a tolerance per channel in engineering units, choose k to meet all of them on a later period, and keep critical low-variance channels outside the compression.

Reconstruction error screens logs for bad-hole intervals

REDUCTION IN PRACTICE

Reconstruction error screens logs for bad-hole intervals

PCA fitted on a reference well; every depth of a new well is scored by its reconstruction error.



Synthetic log suites (reference seed 11, new well seed 23) with an injected washout at 2,120–2,138 m; PCA with k = 4 on the standardized reference well; shading marks depths above the threshold.

Log quality control [3 min]. PCA learns how the logs normally co-vary in a reference well. In the new well, a washout between 2,120 and 2,138 m enlarges the hole: CALI rises by about 2 inches, RHOB reads low, and NPHI and DT read high. Low density and high neutron and sonic readings could each be mistaken for a porous interval; together with the caliper, and with magnitudes that do not match the reference porosity relationships, they are inconsistent, and the reconstruction error rises above the 99th percentile of the reference well.

In this example 53% of the washout depths exceed the threshold, and 0.9% of other depths do, close to the 1% false-alarm rate built into a 99th-percentile threshold. Those scattered false alarms are why flagged intervals go to a petrophysicist for review rather than being deleted automatically.

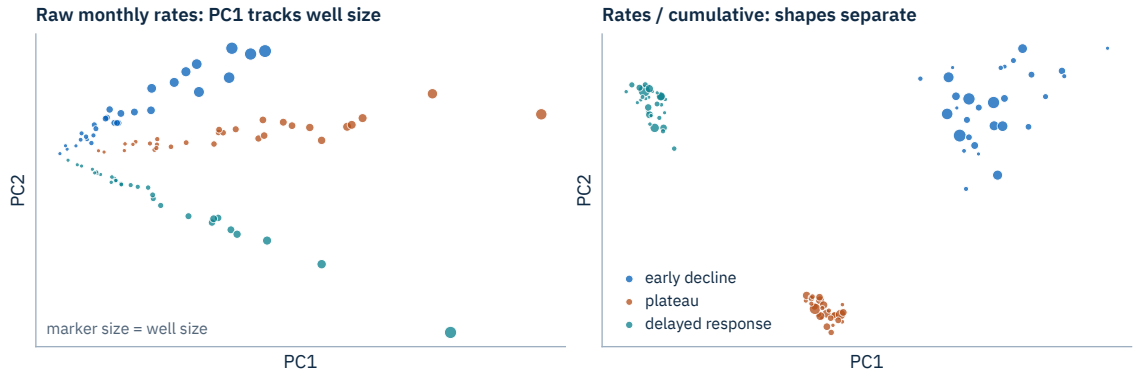
Practical points: the reference well must be clean and geologically comparable; lithologies absent from the reference will also be flagged; and the flag says "inconsistent", not "why". Module 5 covers thresholds and alarm evaluation in depth.

Production maps depend on shape versus amplitude

REDUCTION IN PRACTICE

Production maps depend on shape versus amplitude

Divide each well's rates by its cumulative before PCA when shape, not size, defines similarity.



Synthetic production profiles for 90 wells (seed 13): three shape families, well sizes varying more than tenfold; colours added after fitting.

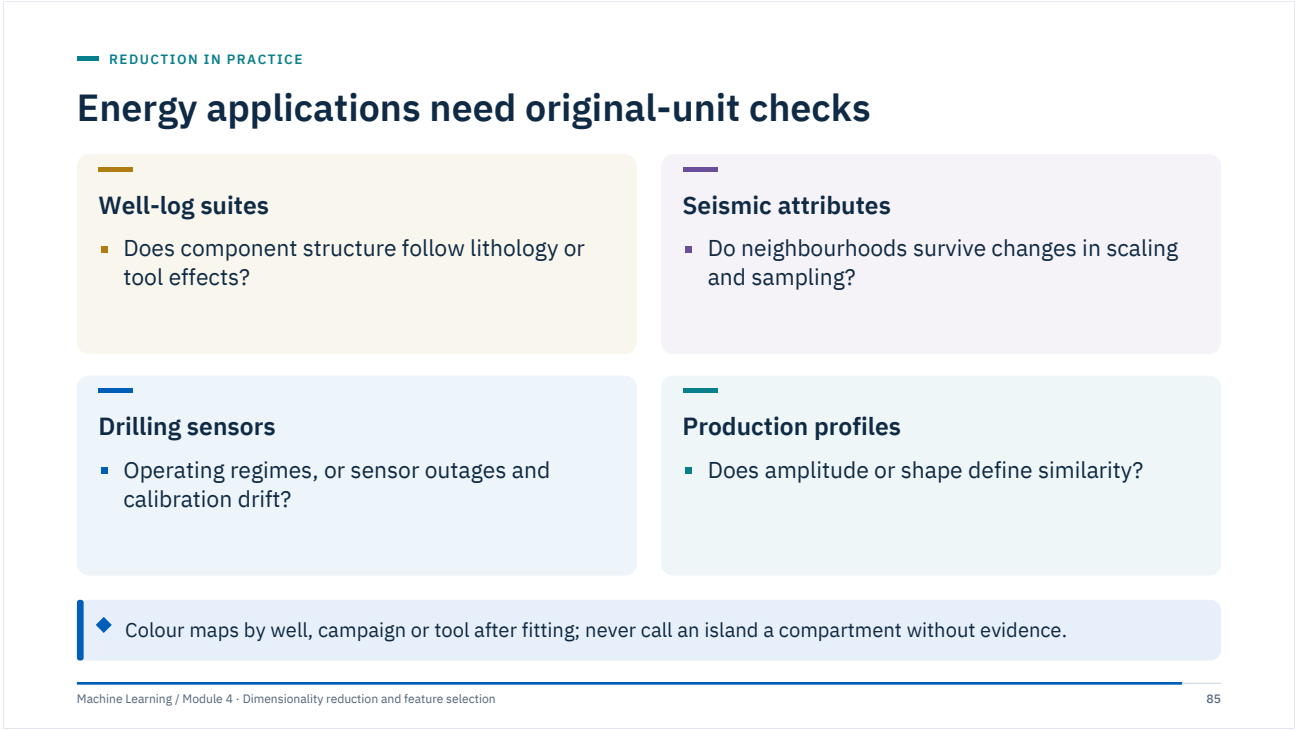
Production pattern maps [3 min]. The synthetic wells follow three shape families (early decline, plateau, delayed response), with sizes varying more than tenfold (about 13-fold between the 5th and 95th percentiles).

Left: PCA on raw monthly rates. PC1 mostly tracks how big the well is (its correlation with log well size is 0.72), and small wells of all three shapes crowd together near the origin. Similarity here means "similar volumes".

Right: the same PCA after dividing each well's rates by its cumulative production. Now the three shape families form separate groups, whatever the well size.

Neither is wrong: the choice depends on the question. For reserves or facility sizing, amplitude matters; for completion design or drive mechanism, shape matters. Decide before fitting, and say which one a map shows. The same choice appears in any similarity map of production data, including t-SNE and UMAP maps.

Energy applications need original-unit checks



One question per data type that must be answered in original units before a reduction result is used [3 min].

Well logs: a component or island may reflect a tool, a vendor, a borehole condition or a correction chain rather than lithology, as in the batch-effect exercise. Seismic attributes: many are strongly correlated by construction, and neighbourhoods can change with the attribute scaling and the sampling grid. Drilling: distinguish operating regimes from sensor outages, stuck values and calibration drift before trusting any structure. Production: decide whether amplitude or shape defines similarity.

Do not label an embedding island a reservoir compartment without supporting evidence. Colour plots by metadata after fitting to diagnose confounding. Compare by well or acquisition campaign to detect batch effects.

Reduction must earn its place in the workflow

KEY TAKEAWAYS

Reduction must earn its place in the workflow

- ✓ High dimensions dilute distances; reduce for a stated goal, not by habit.
- ✓ Select features inside cross-validation, and report how stable the choice is.
- ✓ Standardize mixed units; choose k by task score or held-out error, not a fixed 95%.
- ✓ Kernel PCA, LDA, ICA and NMF change the objective, not just the geometry.
- ✓ Read t-SNE and UMAP maps for neighbours only; check trustworthiness, seeds and metadata.
- ✓ Check reconstructions and maps in original units, per channel and per well.

Read the takeaways and link each one to a slide or lab result from the day [3 min]. The curse of dimensionality: the KNN accuracy falling from 0.94 to 0.61 as noise columns were added. Selection inside cross-validation: 0.88 versus chance on pure noise with 3,000 candidates. Scaling and k: raw-unit PC1 claiming 92% because of psi, and the classifier at chance with the top three components. Other objectives: LDA's d′ of 2.7 where PCA had 0.0; ICA recovering the three sources. Maps: t-SNE sizes and gaps that are not measurements; the batch effect visible only when coloured by well. Original units: the gas channel lost at k = 3 despite about 85% explained variance.

Ask each learner to pick the takeaway most relevant to their next project and write one action on a sticky note. Transition: the exit check.

Five questions check today's core ideas

PRACTICE

5 min Individual

Five questions check today's core ideas

1. Does retaining 95% of variance guarantee retaining target information?
2. Why must SelectKBest sit inside the cross-validation pipeline?
3. Can distances between t-SNE islands be read as physical distances?
4. What must stay fixed when transforming a new well with UMAP?
5. Name one check that a 2-D map's neighbourhoods are trustworthy.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 87

Individual, 3 minutes, then answers [5 min].

1. No. Variance is not relevance: the target can depend on a low-variance direction, as in the classifier that sat at chance with components holding 90% of the variance.

2. Because it uses the target. Ranking on all rows lets validation labels choose the features, which inflated accuracy on pure noise from chance to 0.88 when 3,000 candidates were screened.

3. No. Gaps and island sizes are distorted by adaptive bandwidths and heavy tails; measure distances in the original feature space.

4. The feature schema and order, the fitted imputer and scaler, and the fitted reducer with its settings, all from the training wells. Distribution shift is a separate issue: a consistent transform does not make a new well comparable.

5. Trustworthiness (or continuity) at a stated k; stability across seeds, perplexities or subsamples; colouring by metadata after fitting; and confirming any group in the original logs.

Collect answers to questions 3 and 4 in particular; they are the most common misconceptions.

References and further study

FURTHER STUDY

References and further study

- scikit-learn User Guide: Feature selection; Decomposing signals in components (PCA, kernel PCA, ICA, NMF); Manifold learning; Linear and quadratic discriminant analysis.
- Guyon and Elisseeff (2003), An introduction to variable and feature selection, JMLR 3. Hastie, Tibshirani and Friedman, The Elements of Statistical Learning, 2nd ed., sections 2.5 and 7.10.2, chapter 14.
- Jolliffe and Cadima (2016), Principal component analysis: a review and recent developments, Phil. Trans. R. Soc. A 374.
- Tenenbaum, de Silva and Langford (2000), Science 290; Roweis and Saul (2000), Science 290; van der Maaten and Hinton (2008), Visualizing data using t-SNE, JMLR 9; McInnes, Healy and Melville (2018), UMAP, arXiv:1802.03426.
- Hyvärinen and Oja (2000), Independent component analysis, Neural Networks 13; Lee and Seung (1999), Nature 401; Wattenberg, Viégas and Johnson (2016), How to use t-SNE effectively, Distill.

Machine Learning / Module 4 · Dimensionality reduction and feature selection 88

Suggested reading paths [1 min]. For practitioners: the scikit-learn user guide pages, which include runnable examples for every method used today, and the Distill article on reading t-SNE, which has interactive examples of the distortions seen in Part 5.

For depth: Guyon and Elisseeff for feature selection; Hastie, Tibshirani and Friedman section 2.5 (local methods in high dimensions), section 7.10.2 (the wrong and right way to do cross-validation) and chapter 14 (PCA, NMF, ICA, multidimensional scaling and nonlinear dimension reduction); Jolliffe and Cadima for a modern PCA review.

Original papers: Isomap and LLE (both Science, 2000), t-SNE (2008) and UMAP (2018); Hyvärinen and Oja for ICA; Lee and Seung for NMF.

Reference: https://scikit-learn.org/stable/modules/feature_selection.html ; https://scikit-learn.org/stable/modules/decomposition.html ; https://scikit-learn.org/stable/modules/manifold.html ; https://umap-learn.readthedocs.io/ ; https://distill.pub/2016/misread-tsne/